

Assignment No. 02

Course Title: CS-426L Parallel and Distributed Computing Lab

TITLE OF THE ASSIGNMENT

"From Custom Linear Structures and Multi-Threading to Fork/Join Parallelism and Cache-Aware Optimization in Java"

SUBMITTED BY:

Name: Usama Arshad

Student ID: 247141

Semester: 5th

Section: BS-CYS-F24-A

SUBMITTED TO:

Instructor Name: Mr. Qaiser Ali

Date of Submission: October 19, 2026

AIR UNIVERSITY, KHARIAN CAMPUS

DEPARTMENT OF Computer Science

GitHub Repository Link: *https://github.com/usama-dot-ai/University-Projects/tree/main/Semester-5/Parallel_and_Distributed_Computing/Assignments/Assignment_02_Advanced_OOP_Data_Structures_Concurrency_and_Cache_Locality*

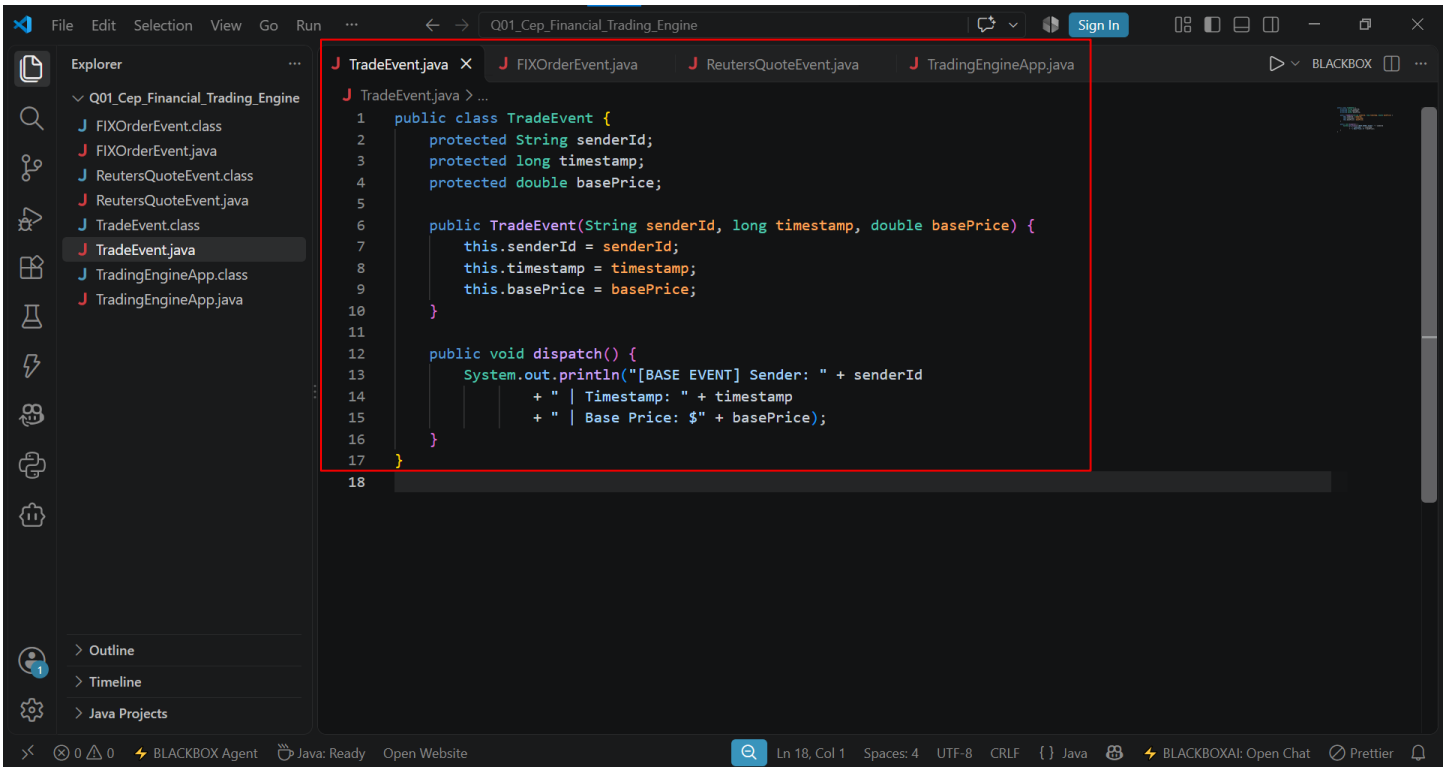
Question 1: Trade Event Dispatcher (Inheritance & Method Overriding)

1. Approach / Explanation

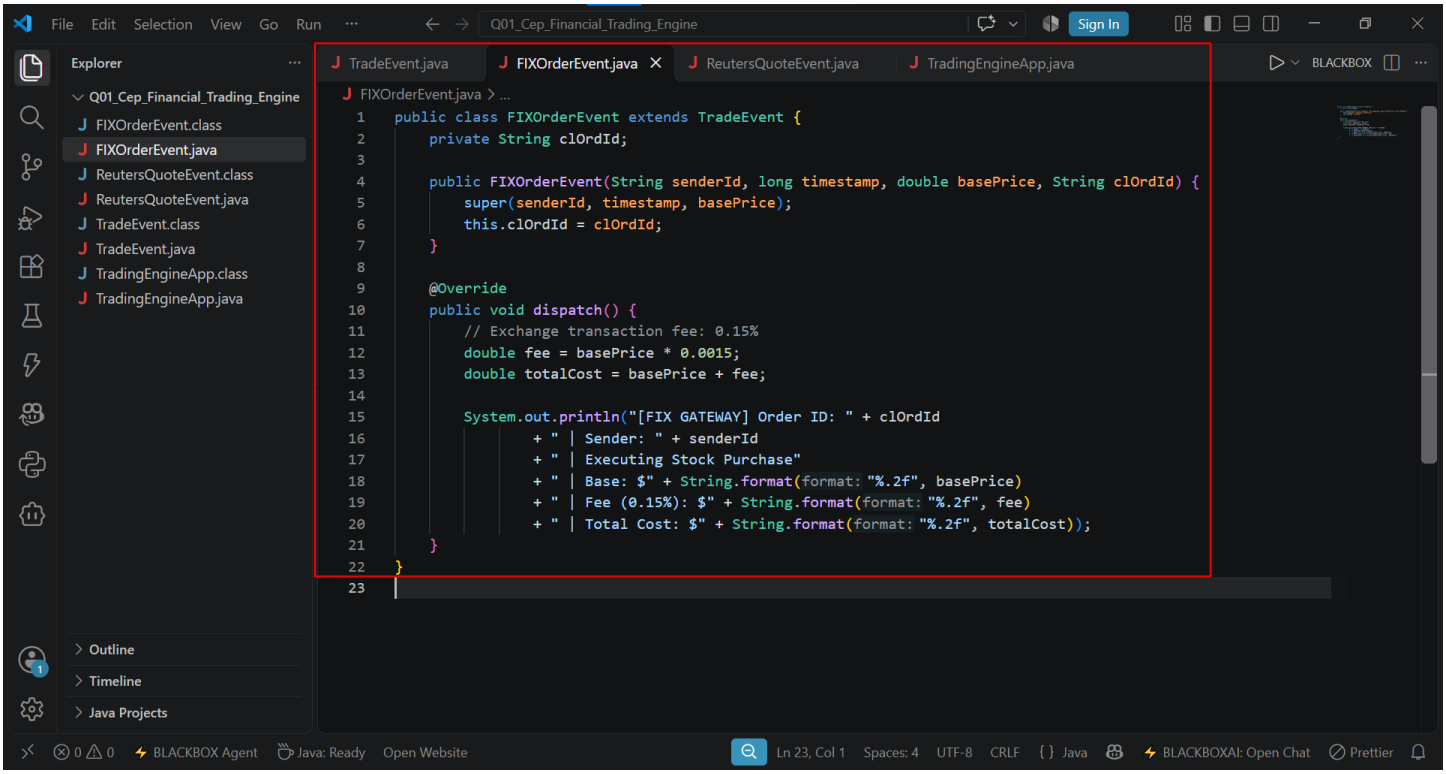
I created four Java files (TradeEvent.java, FIXOrderEvent.java, ReutersQuoteEvent.java, and TradingEngineApp.java) to build a financial trading engine simulation. I structured TradeEvent as a superclass with protected variables (senderId, timestamp, basePrice) and a generic dispatch method, while FIXOrderEvent and ReutersQuoteEvent extended it as subclasses. In the subclasses, I used constructors with super(...) to initialize shared fields. I overrode the dispatch method to apply specific logic, such as calculating a 0.15% exchange fee or a bid-ask spread. In the main app, I instantiated these subclasses using superclass references to demonstrate runtime polymorphism. Running the program compiled all classes and printed distinct event outputs, verifying that the correct overridden methods executed and produced accurate mathematical results.

2. Code

File: *TradeEvent.java*



File: *FIXOrderEvent.java*



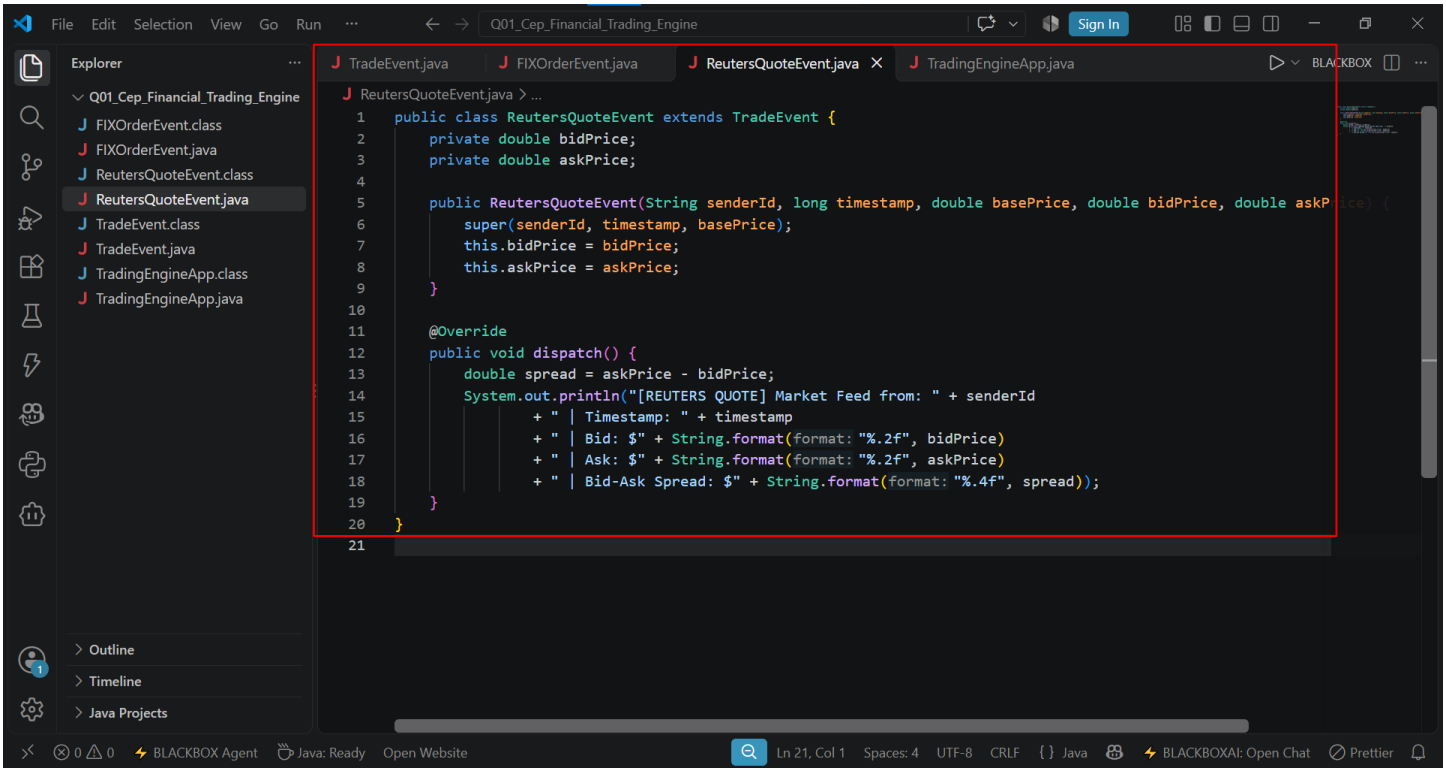

```java
public class FIXOrderEvent extends TradeEvent {
    private String clOrdId;

    public FIXOrderEvent(String senderId, long timestamp, double basePrice, String clOrdId) {
        super(senderId, timestamp, basePrice);
        this.clOrdId = clOrdId;
    }

    @Override
    public void dispatch() {
        // Exchange transaction fee: 0.15%
        double fee = basePrice * 0.0015;
        double totalCost = basePrice + fee;

        System.out.println("[FIX GATEWAY] Order ID: " + clOrdId
                + " | Sender: " + senderId
                + " | Executing Stock Purchase"
                + " | Base: $" + String.format(format: "%.2f", basePrice)
                + " | Fee (0.15%): $" + String.format(format: "%.2f", fee)
                + " | Total Cost: $" + String.format(format: "%.2f", totalCost));
    }
}
```

File: *ReutersQuoteEvent.java*

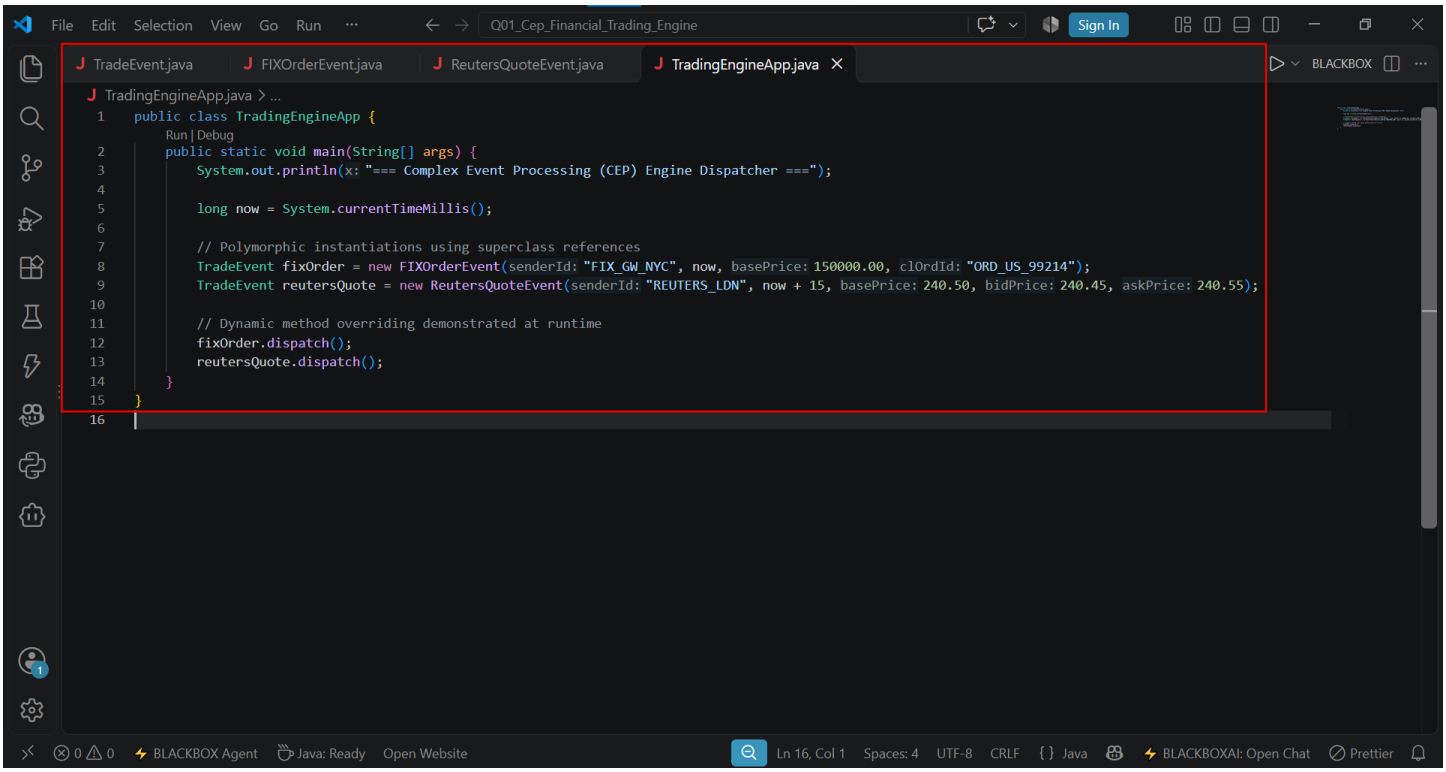



```java
public class ReutersQuoteEvent extends TradeEvent {
    private double bidPrice;
    private double askPrice;

    public ReutersQuoteEvent(String senderId, long timestamp, double basePrice, double bidPrice, double askP
        super(senderId, timestamp, basePrice);
        this.bidPrice = bidPrice;
        this.askPrice = askPrice;
    }

    @Override
    public void dispatch() {
        double spread = askPrice - bidPrice;
        System.out.println("[REUTERS QUOTE] Market Feed from: " + senderId
                + " | Timestamp: " + timestamp
                + " | Bid: $" + String.format(format: "%.2f", bidPrice)
                + " | Ask: $" + String.format(format: "%.2f", askPrice)
                + " | Bid-Ask Spread: $" + String.format(format: "%.4f", spread));
    }
}
```

File: *TradingEngineApp.java*



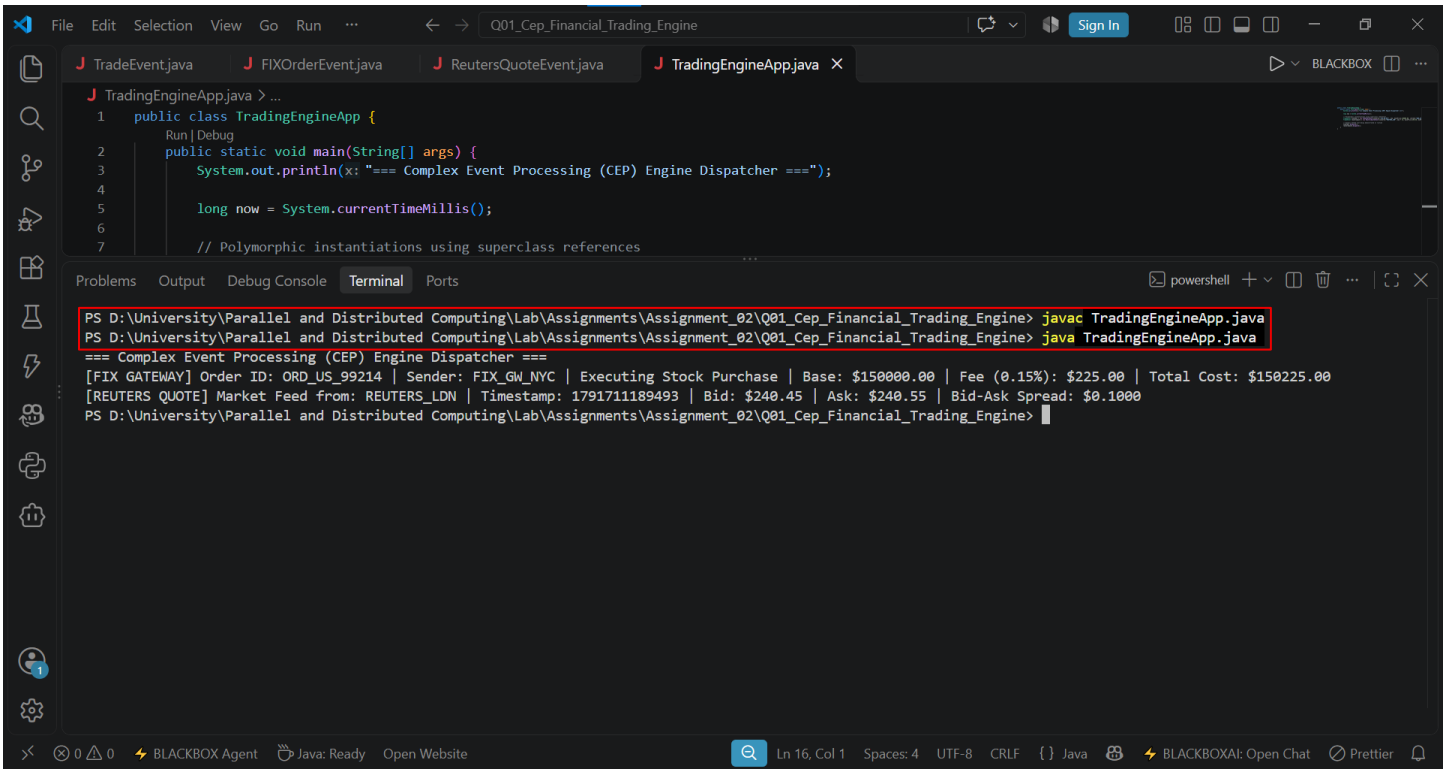
```java
public class TradingEngineApp {
    public static void main(String[] args) {
        System.out.println("=== Complex Event Processing (CEP) Engine Dispatcher ===");

        long now = System.currentTimeMillis();

        // Polymorphic instantiations using superclass references
        TradeEvent fixOrder = new FIXOrderEvent(senderId: "FIX_GW_NYC", now, basePrice: 150000.00, clOrdId: "ORD_US_99214");
        TradeEvent reutersQuote = new ReutersQuoteEvent(senderId: "REUTERS_LDN", now + 15, basePrice: 240.50, bidPrice: 240.45, askPrice: 240.55);

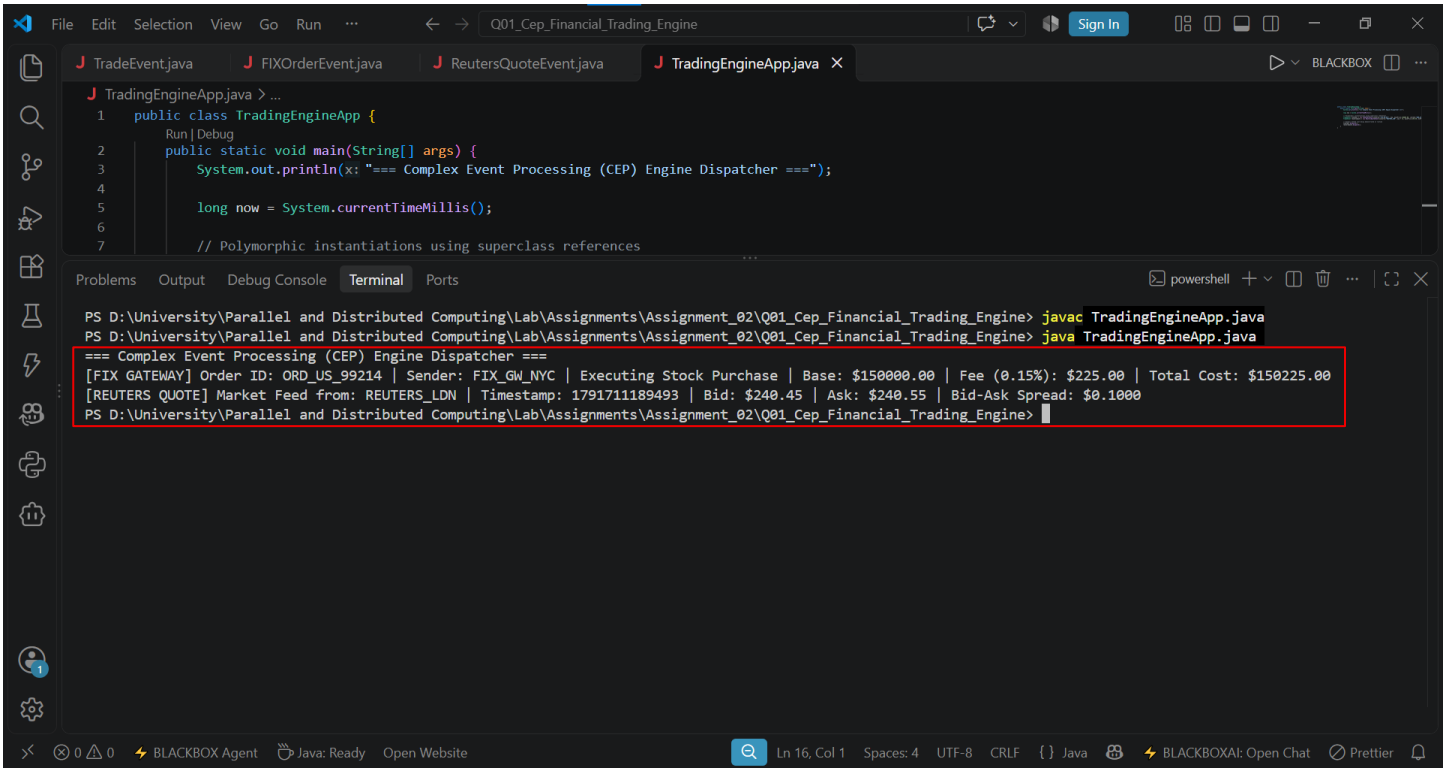
        // Dynamic method overriding demonstrated at runtime
        fixOrder.dispatch();
        reutersQuote.dispatch();
    }
}
```

3. Compilation & Execution Screenshot



```
PS D:\University\Parallel and Distributed Computing\Lab\Assignments\Assignment_02\Q01_Cep_Financial_Trading_Engine> javac TradingEngineApp.java
PS D:\University\Parallel and Distributed Computing\Lab\Assignments\Assignment_02\Q01_Cep_Financial_Trading_Engine> java TradingEngineApp.java
=== Complex Event Processing (CEP) Engine Dispatcher ===
[FIX GATEWAY] Order ID: ORD_US_99214 | Sender: FIX_GW_NYC | Executing Stock Purchase | Base: $150000.00 | Fee (0.15%): $225.00 | Total Cost: $150225.00
[REUTERS QUOTE] Market Feed from: REUTERS_LDN | Timestamp: 1791711189493 | Bid: $240.45 | Ask: $240.55 | Bid-Ask Spread: $0.1000
PS D:\University\Parallel and Distributed Computing\Lab\Assignments\Assignment_02\Q01_Cep_Financial_Trading_Engine>
```

4. Output Screenshot



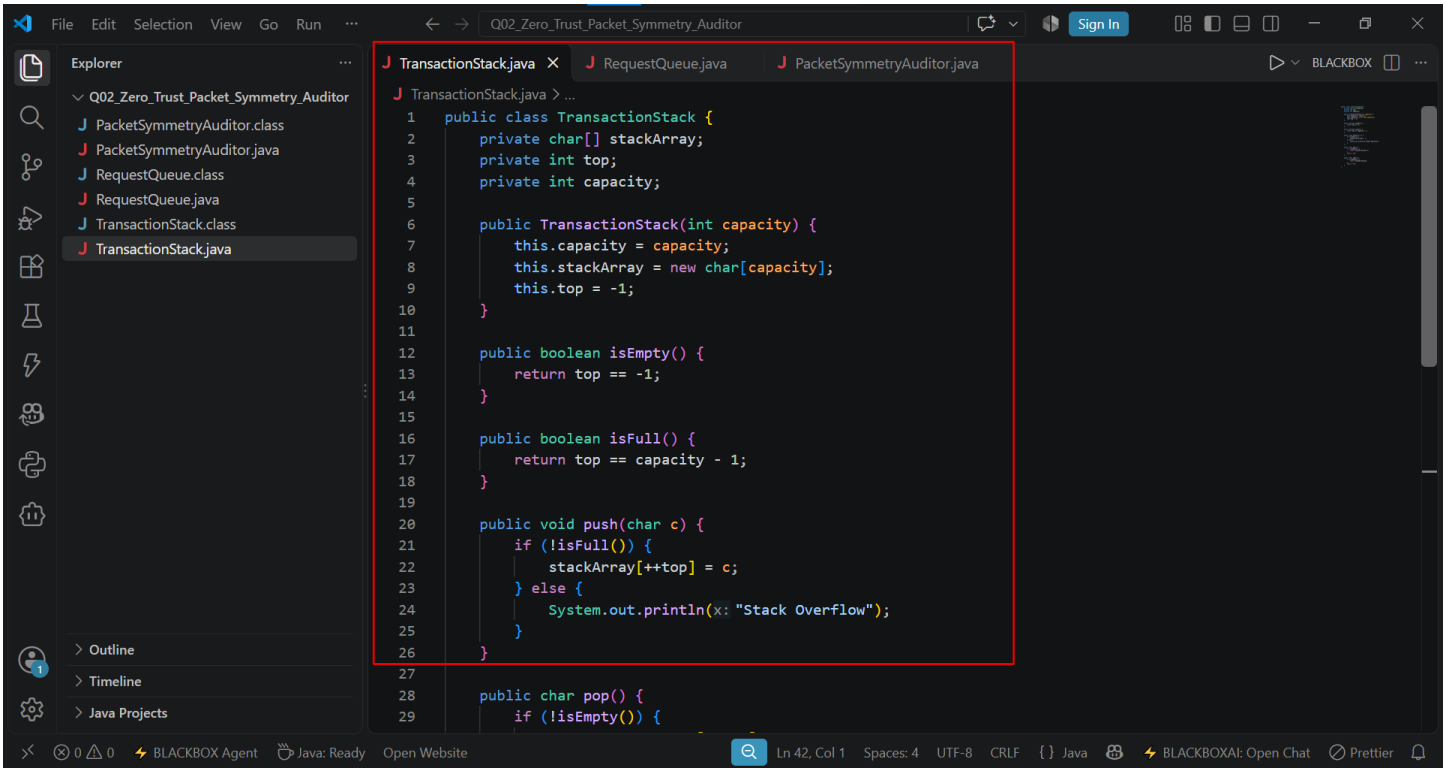
Question 2: Packet Symmetry Auditor (Array-Based Stack & Queue)

1. Approach / Explanation

I created TransactionStack.java, RequestQueue.java, and PacketSymmetryAuditor.java to build a packet symmetry auditor that checks if a string payload reads the same forwards and backwards. I implemented TransactionStack using a char array with a top index for LIFO operations (push, pop), and RequestQueue using a circular char array with front, rear, and size variables plus modulo arithmetic ((rear + 1) % capacity) for FIFO operations (enqueue, dequeue). In PacketSymmetryAuditor, I wrote auditPacketSymmetry(String payload) to load each character into both structures using a for loop, then compare them in a while loop by popping from the stack (reverse order) and dequeuing from the queue (original order). The terminal output verified the logic: palindromic payloads like "REVIVER" and "10011001" returned true, while asymmetric ones like "TXN_104_01_NXT" and "SECURITY_GATEWAY" returned false, confirming the stack and queue correctly compared the sequences.

2. Code

File: *TransactionStack.java*




```java
public class TransactionStack {
    private char[] stackArray;
    private int top;
    private int capacity;

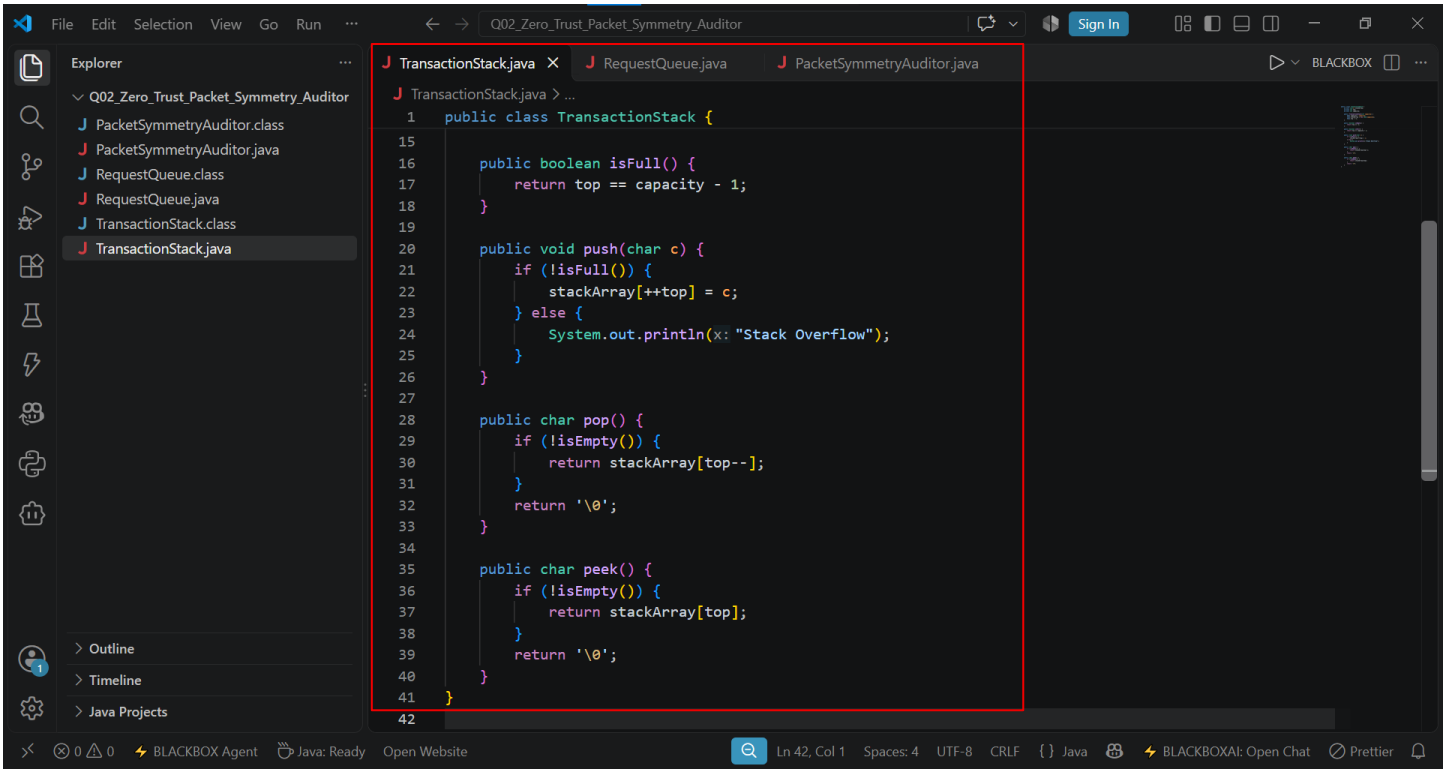
    public TransactionStack(int capacity) {
        this.capacity = capacity;
        this.stackArray = new char[capacity];
        this.top = -1;
    }

    public boolean isEmpty() {
        return top == -1;
    }

    public boolean isFull() {
        return top == capacity - 1;
    }

    public void push(char c) {
        if (!isFull()) {
            stackArray[++top] = c;
        } else {
            System.out.println(x: "Stack Overflow");
        }
    }

    public char pop() {
        if (!isEmpty()) {
```




```java
public class TransactionStack {

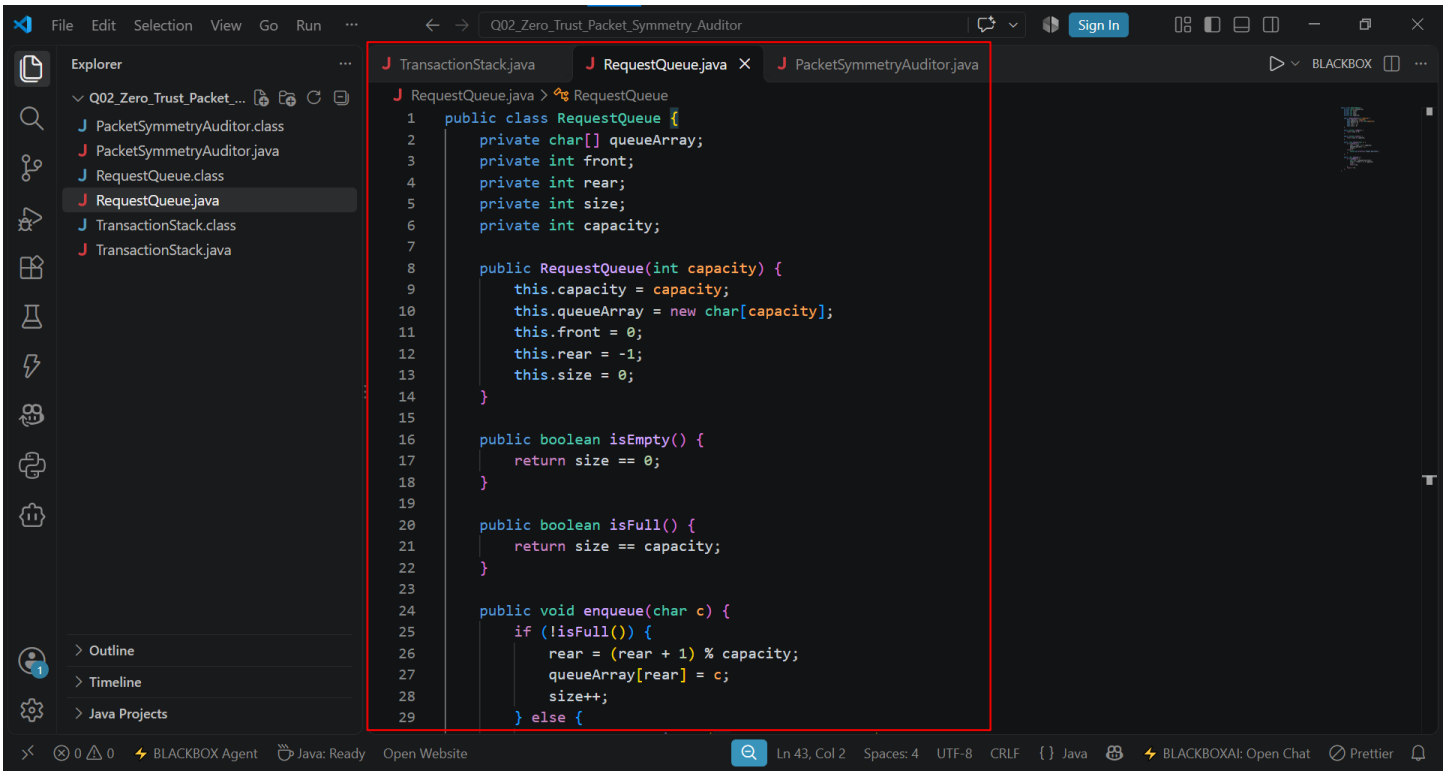
    public boolean isFull() {
        return top == capacity - 1;
    }

    public void push(char c) {
        if (!isFull()) {
            stackArray[++top] = c;
        } else {
            System.out.println(x: "Stack Overflow");
        }
    }

    public char pop() {
        if (!isEmpty()) {
            return stackArray[top--];
        }
        return '\0';
    }

    public char peek() {
        if (!isEmpty()) {
            return stackArray[top];
        }
        return '\0';
    }
}
```

File: *RequestQueue.java*



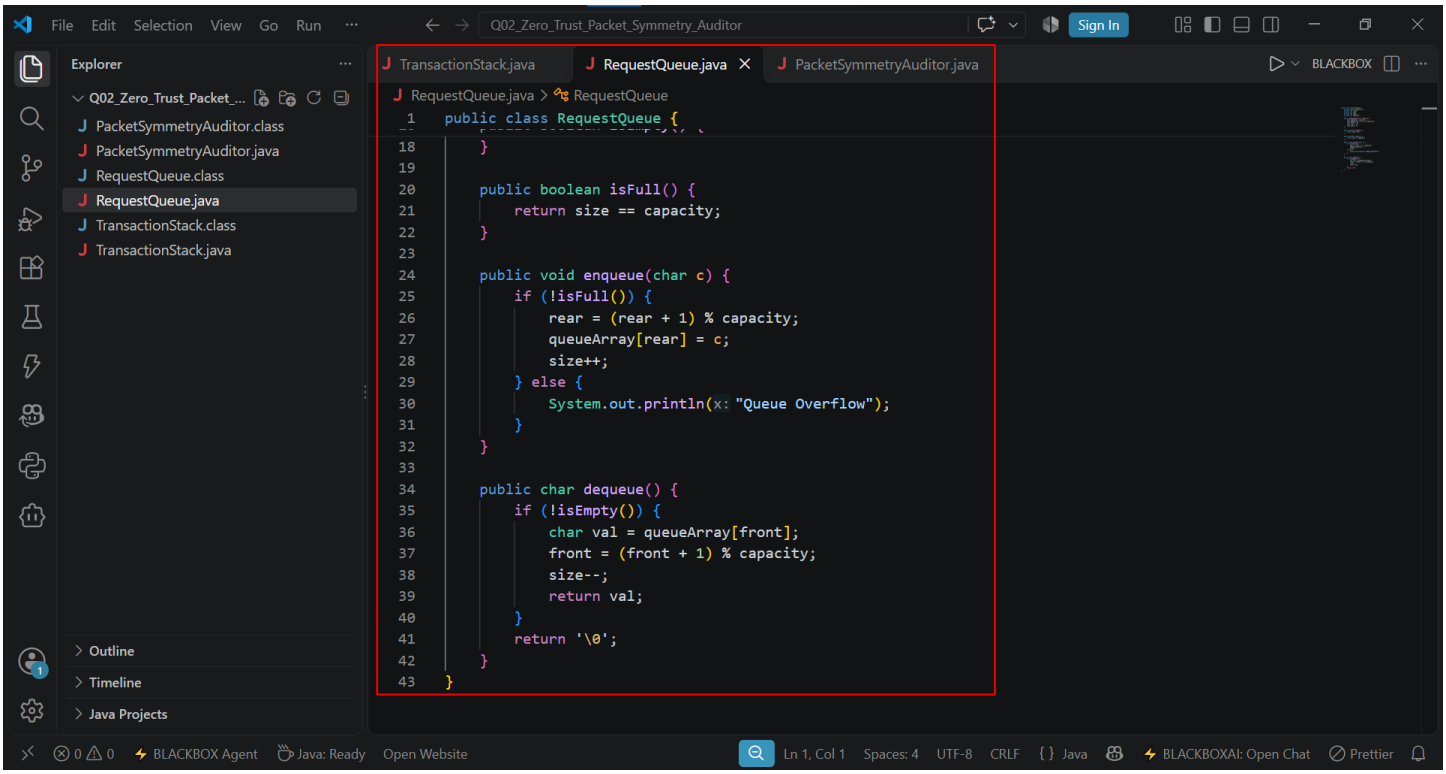

```java
public class RequestQueue {
    private char[] queueArray;
    private int front;
    private int rear;
    private int size;
    private int capacity;

    public RequestQueue(int capacity) {
        this.capacity = capacity;
        this.queueArray = new char[capacity];
        this.front = 0;
        this.rear = -1;
        this.size = 0;
    }

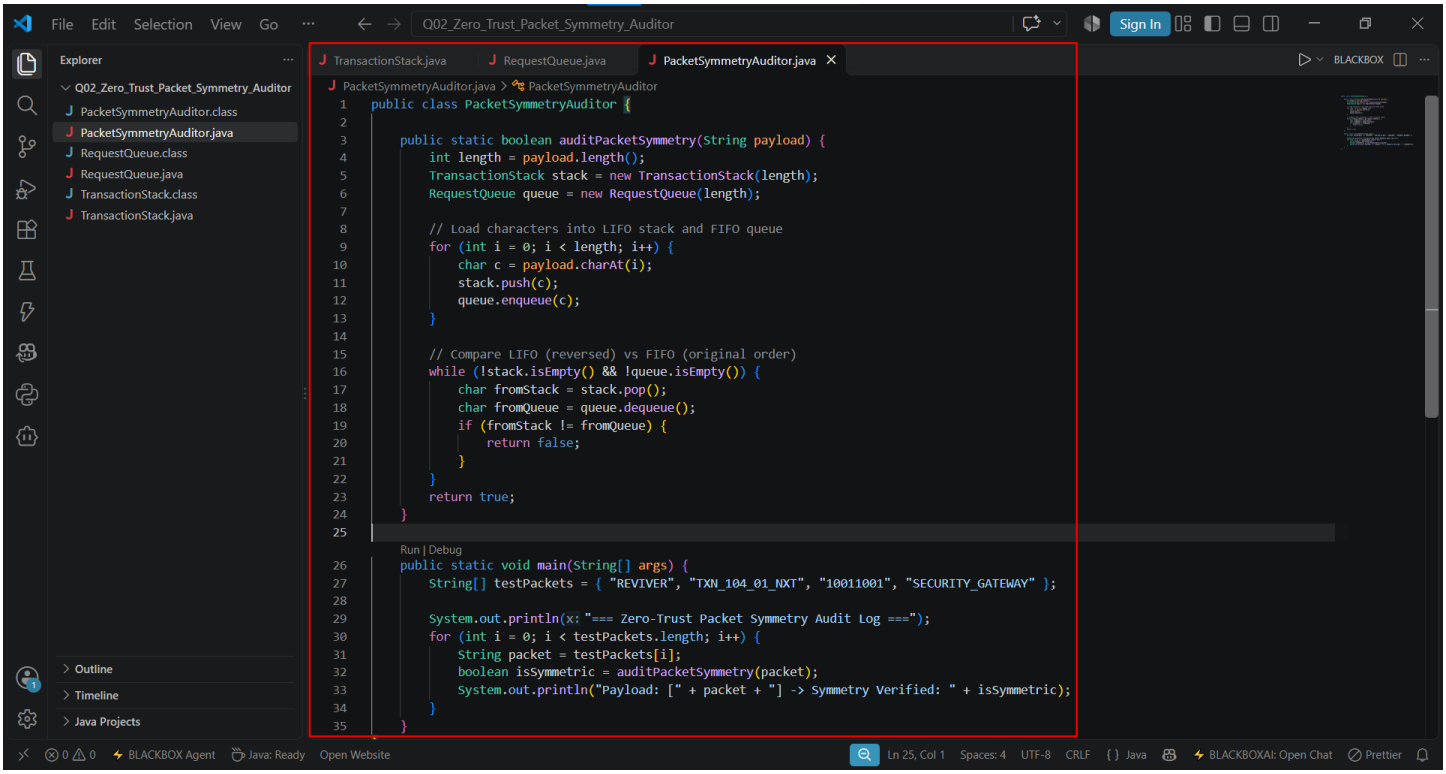
    public boolean isEmpty() {
        return size == 0;
    }

    public boolean isFull() {
        return size == capacity;
    }

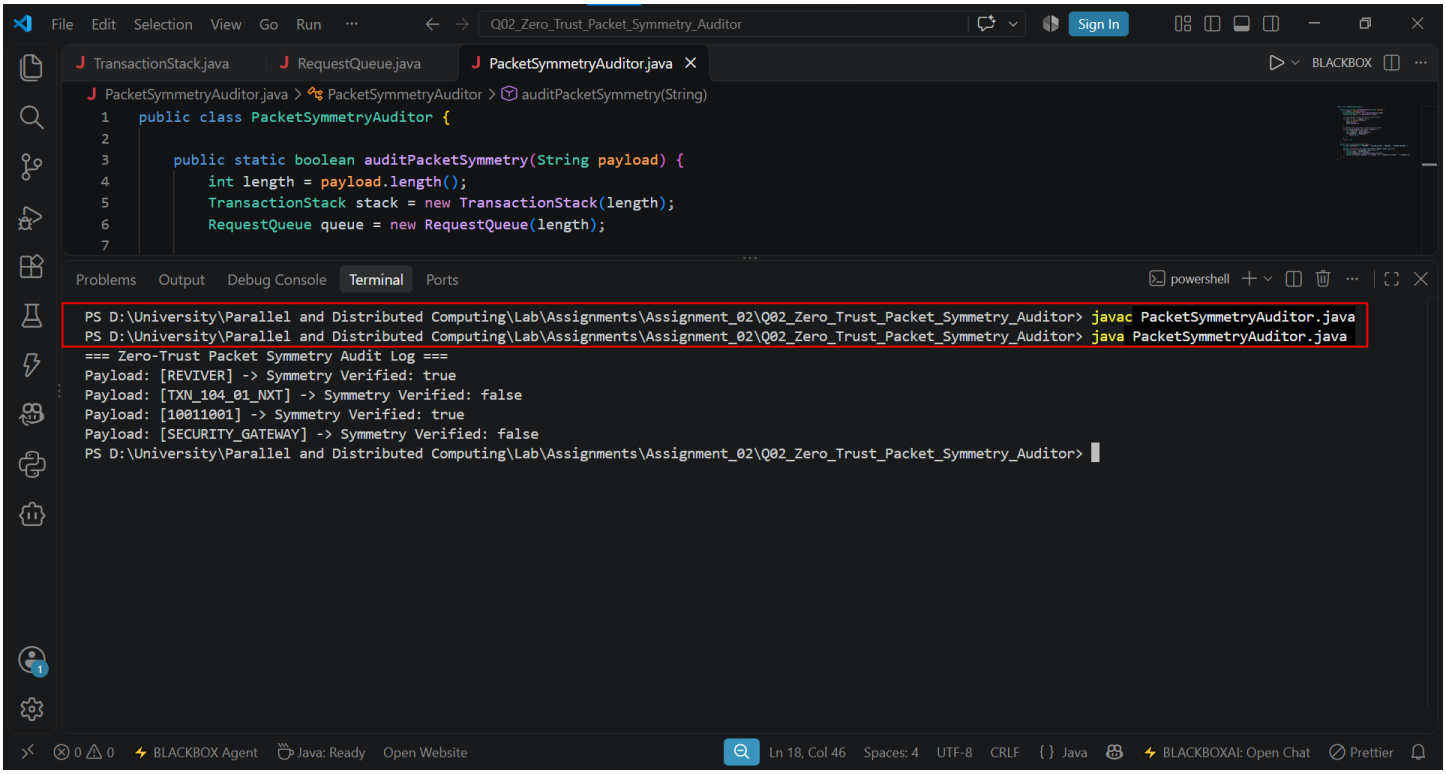
    public void enqueue(char c) {
        if (!isFull()) {
            rear = (rear + 1) % capacity;
            queueArray[rear] = c;
            size++;
        } else {
```



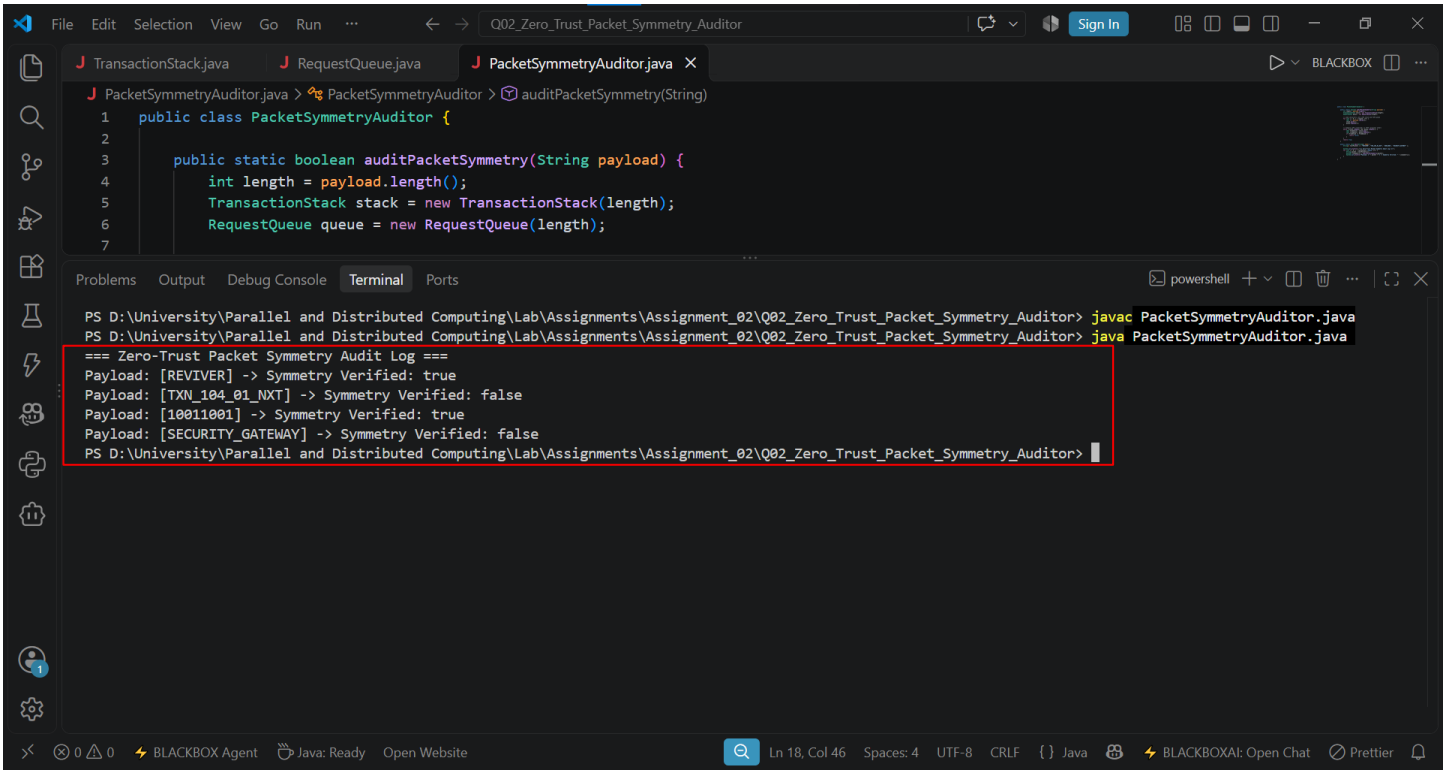
File: *PacketSymmetryAuditor.java*



3. Compilation & Execution Screenshot



4. Output Screenshot



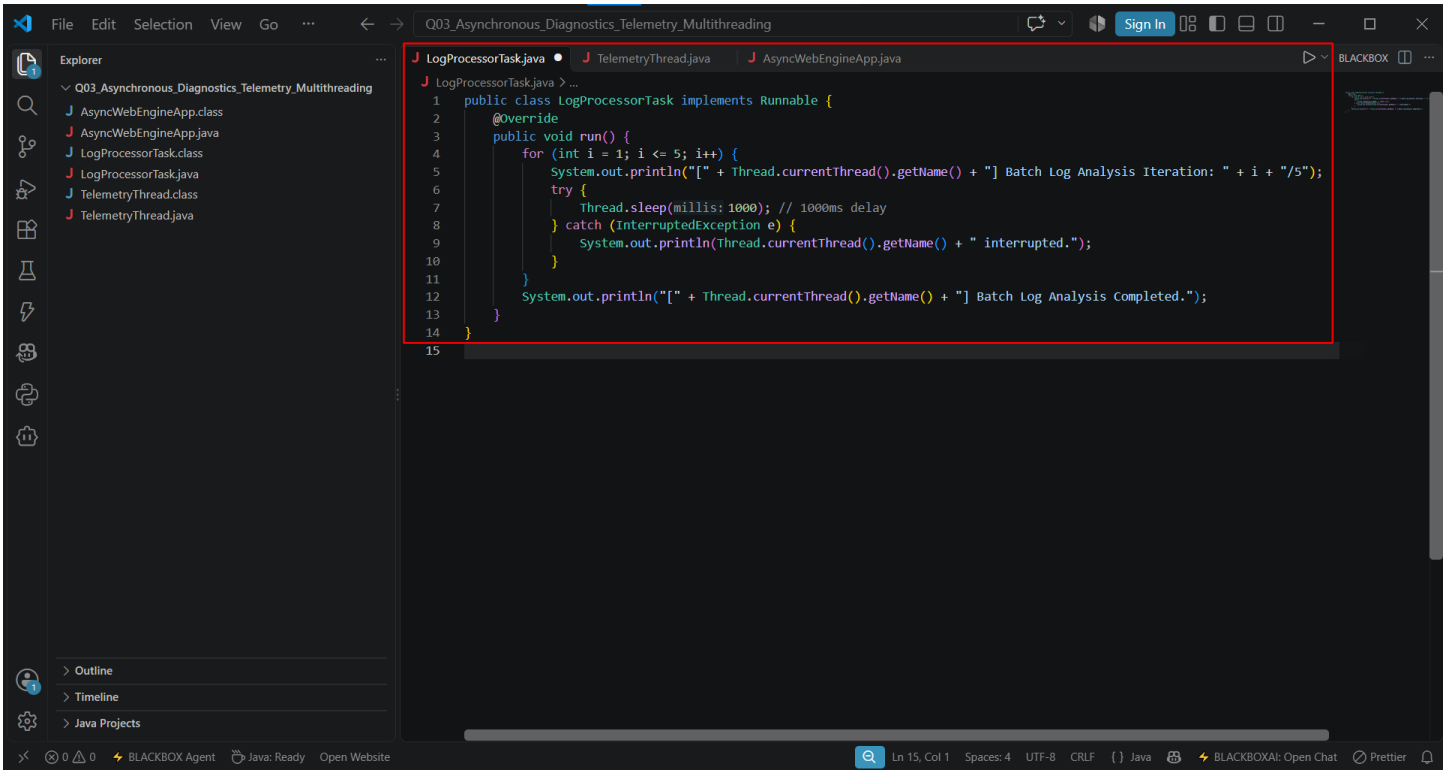
Question 3: Multithreaded Telemetry Engine (Runnable & Thread)

1. Approach / Explanation

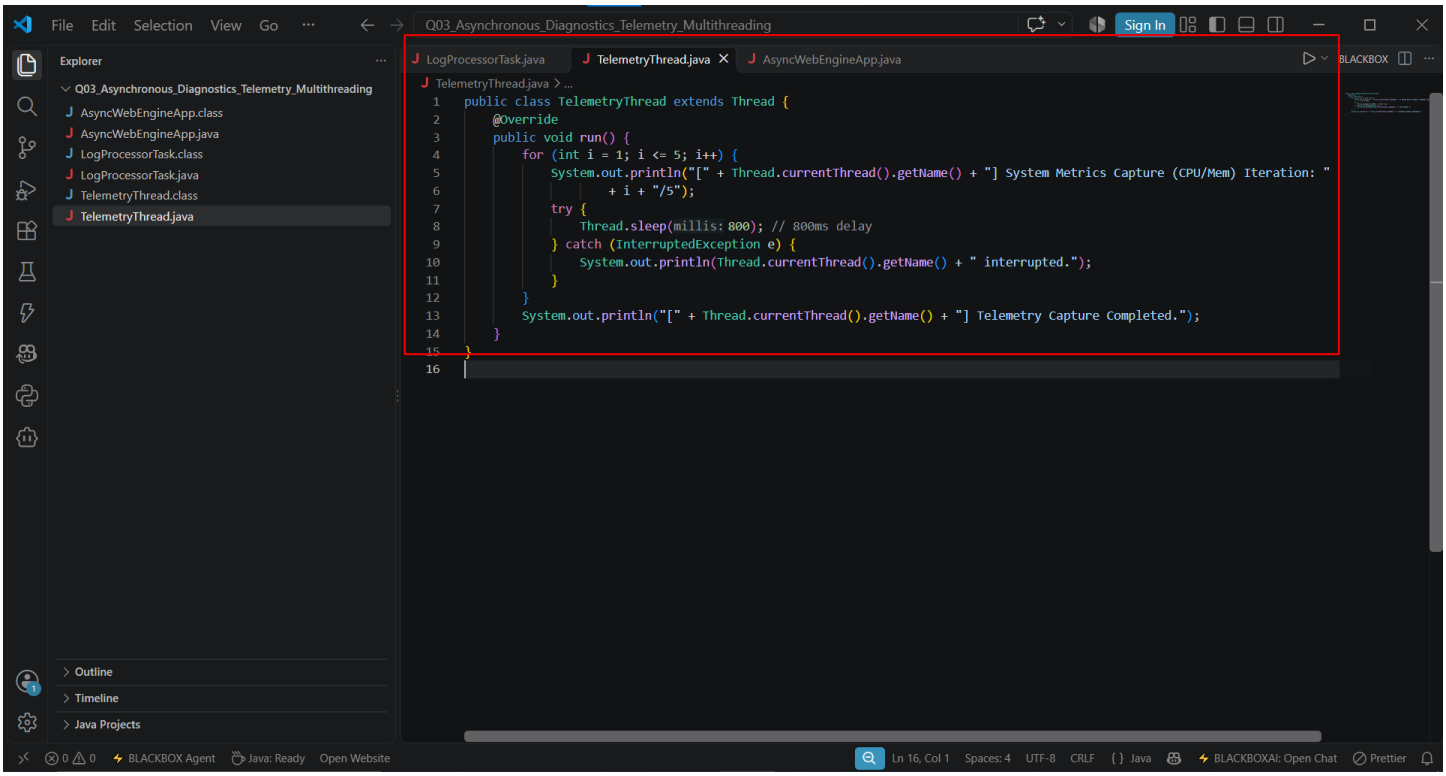
I created three files (LogProcessorTask.java, TelemetryThread.java, and AsyncWebEngineApp.java) to demonstrate multithreading and asynchronous execution. I implemented LogProcessorTask using the Runnable interface and TelemetryThread by extending Thread, using a for loop in both run methods to run five iterations of simulated work with Thread.sleep() delays of 1000ms and 800ms, respectively. In the main application, I instantiated both workers, assigned them specific names, and started them concurrently while checking their initial isAlive() status. To ensure the main thread waited for the background tasks to finish, I used Thread.join() inside a try-catch block. The terminal output verified correct execution by displaying interleaved, asynchronous progress logs from both threads, followed by the main thread's final exit message only after the background workers completed.

2. Code

File: *LogProcessorTask.java*

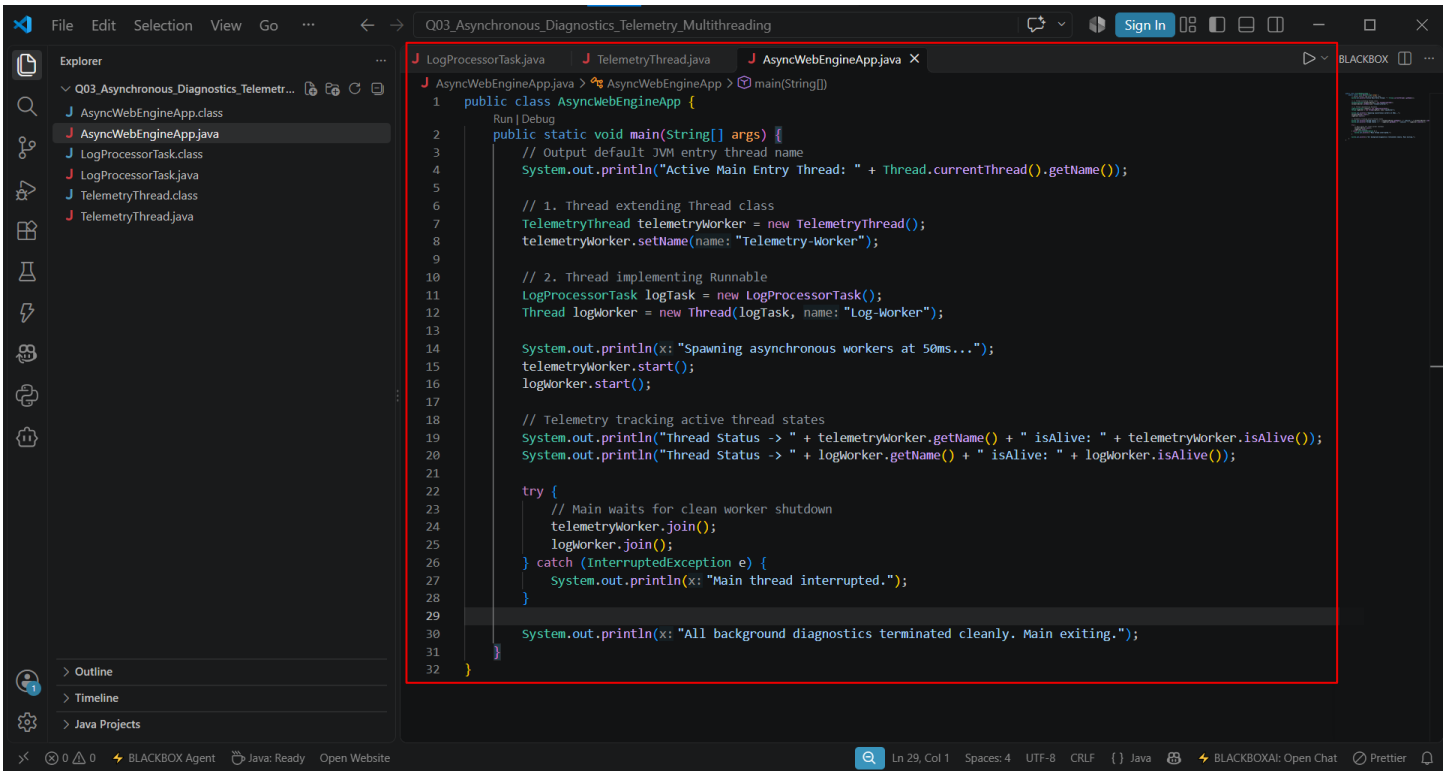


File: *TelemetryThread.java*



```java
public class TelemetryThread extends Thread {
    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("[" + Thread.currentThread().getName() + "] System Metrics Capture (CPU/Mem) Iteration: "
                    + i + "/5");
            try {
                Thread.sleep(millis: 800); // 800ms delay
            } catch (InterruptedException e) {
                System.out.println(Thread.currentThread().getName() + " interrupted.");
            }
        }
        System.out.println("[" + Thread.currentThread().getName() + "] Telemetry Capture Completed.");
    }
}
```

File: *AsyncWebEngineApp.java*



```java
public class AsyncWebEngineApp {
    Run | Debug
    public static void main(String[] args) {
        // Output default JVM entry thread name
        System.out.println("Active Main Entry Thread: " + Thread.currentThread().getName());

        // 1. Thread extending Thread class
        TelemetryThread telemetryWorker = new TelemetryThread();
        telemetryWorker.setName(name: "Telemetry-Worker");

        // 2. Thread implementing Runnable
        LogProcessorTask logTask = new LogProcessorTask();
        Thread logWorker = new Thread(logTask, name: "Log-Worker");

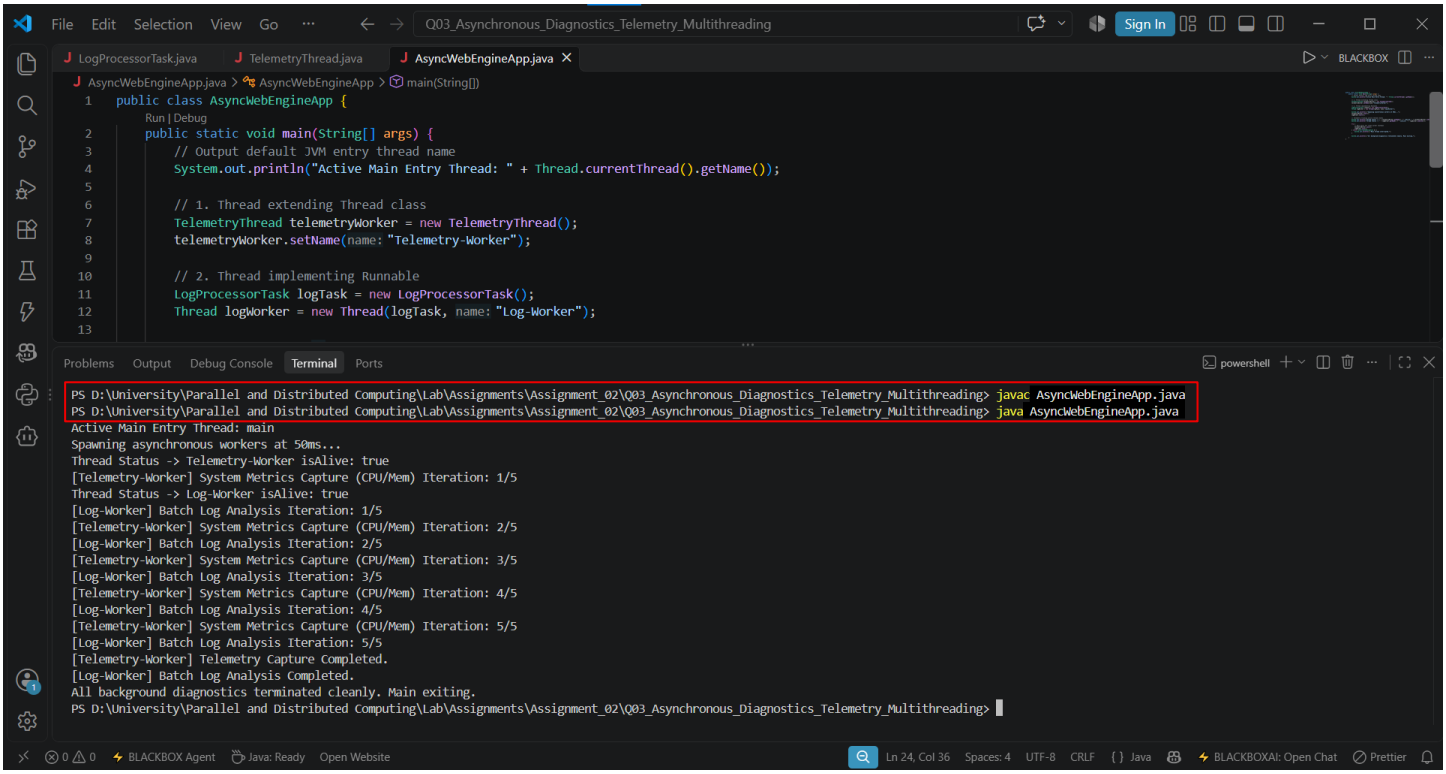
        System.out.println(x: "Spawning asynchronous workers at 50ms...");
        telemetryWorker.start();
        logWorker.start();

        // Telemetry tracking active thread states
        System.out.println("Thread Status -> " + telemetryWorker.getName() + " isAlive: " + telemetryWorker.isAlive());
        System.out.println("Thread Status -> " + logWorker.getName() + " isAlive: " + logWorker.isAlive());

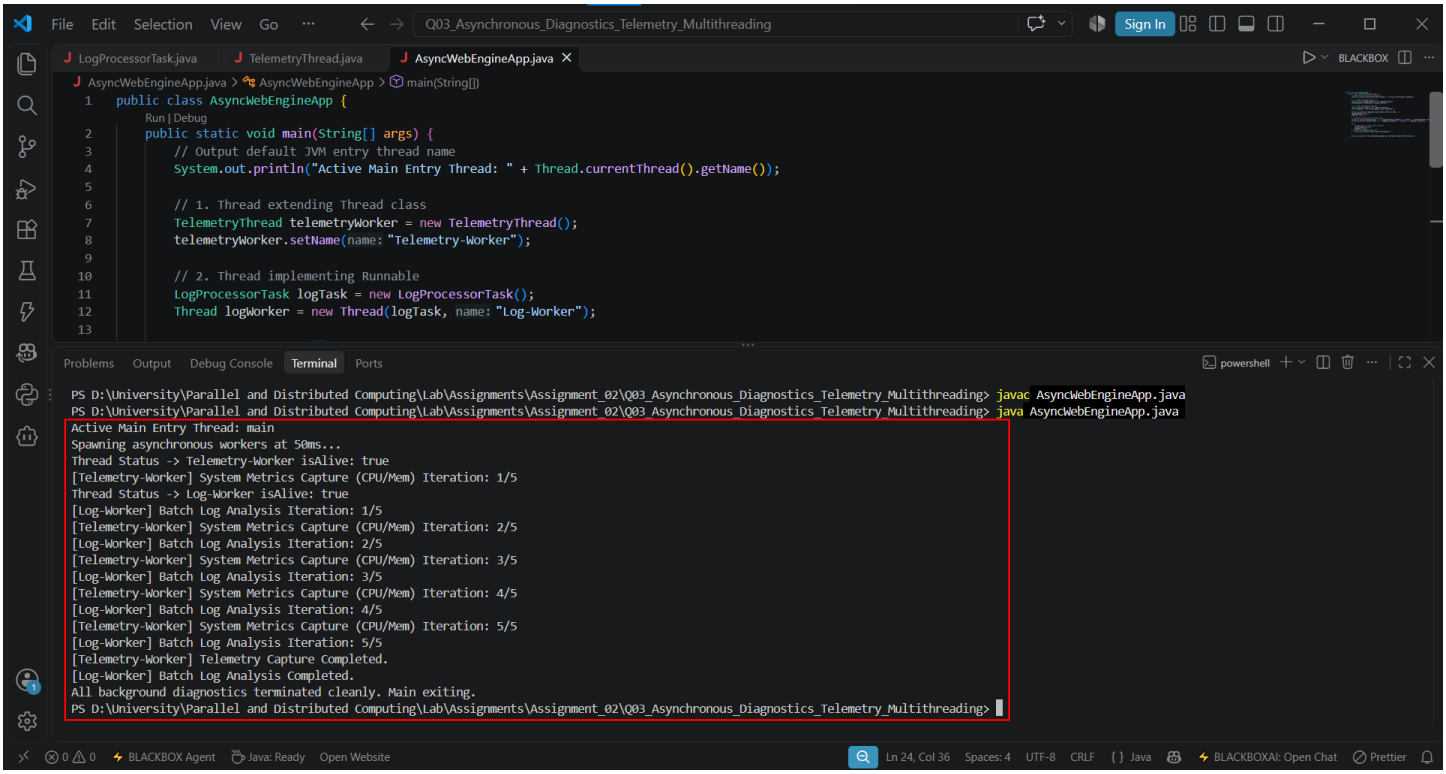
        try {
            // Main waits for clean worker shutdown
            telemetryWorker.join();
            logWorker.join();
        } catch (InterruptedException e) {
            System.out.println(x: "Main thread interrupted.");
        }

        System.out.println(x: "All background diagnostics terminated cleanly. Main exiting.");
    }
}
```

3. Compilation & Execution Screenshot



4. Output Screenshot



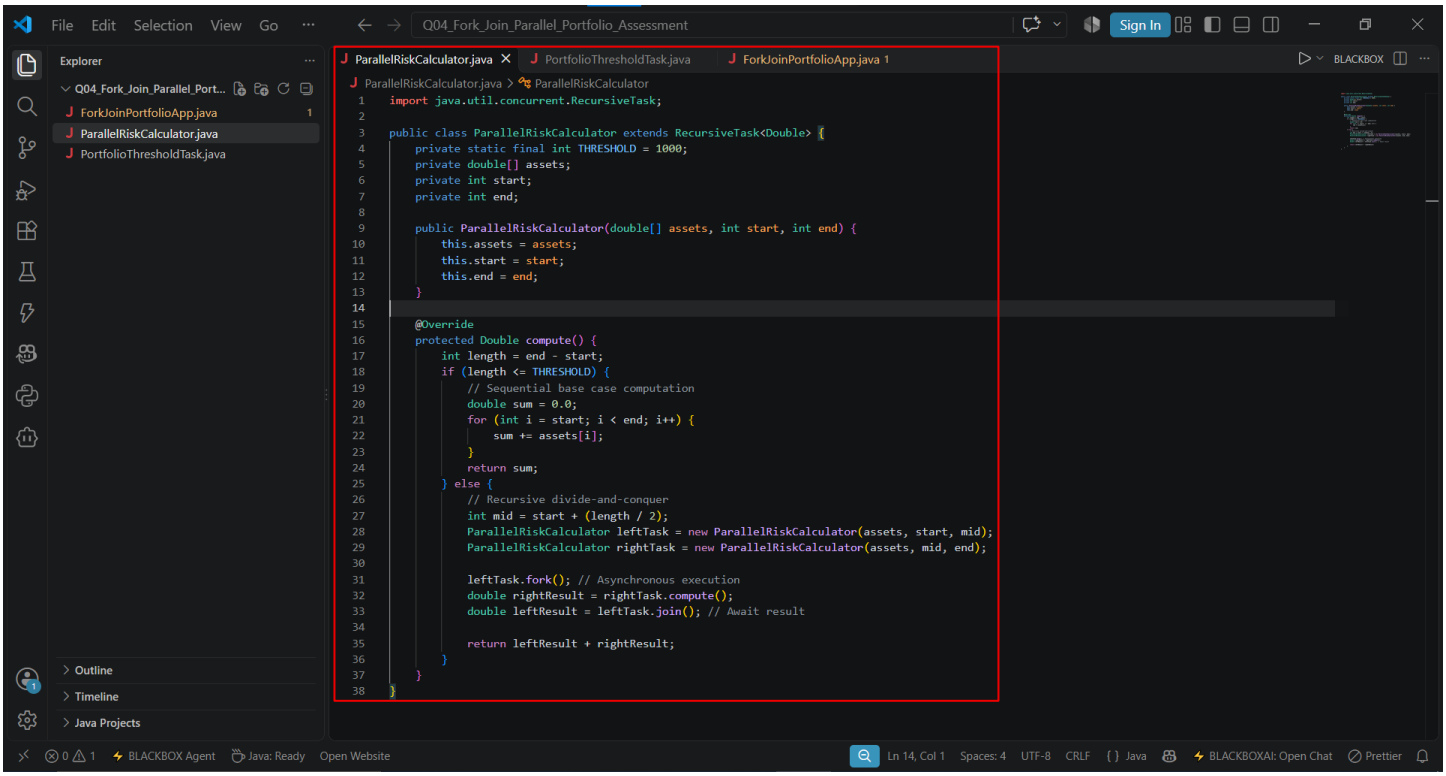
Question 4: Parallel Portfolio Assessment (Fork/Join Framework)

1. Approach / Explanation

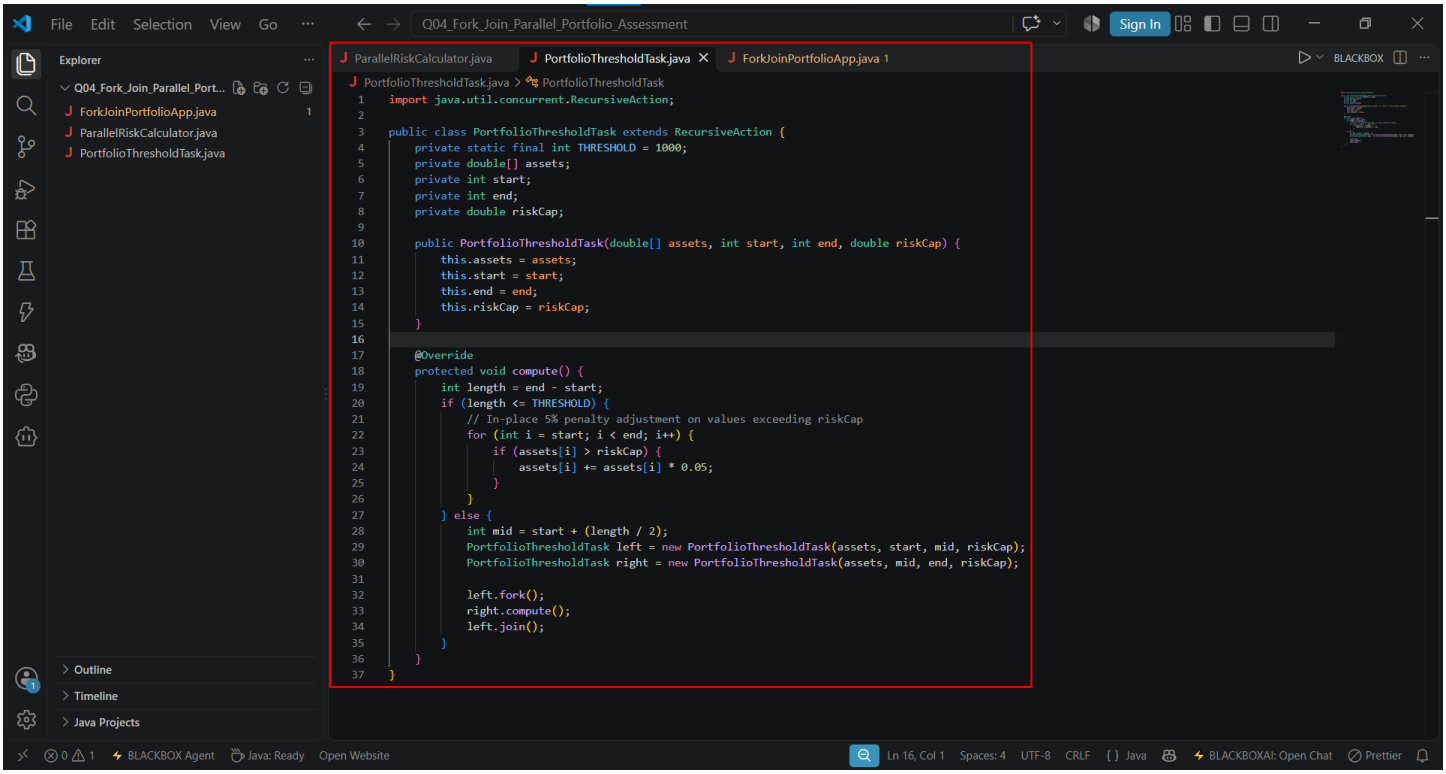
I created three files (ParallelRiskCalculator.java, PortfolioThresholdTask.java, and ForkJoinPortfolioApp.java) to simulate a parallel financial portfolio assessment using the Fork/Join framework. I structured ParallelRiskCalculator to extend RecursiveTask<Double> and return the total risk sum, and PortfolioThresholdTask to extend RecursiveAction and apply a 5% penalty in place to assets exceeding the risk cap. Both classes used a THRESHOLD of 1000 to recursively split the array in half until the segment size was small enough for sequential processing via a for loop, utilizing fork() and join() to execute tasks concurrently. In the main application, I initialized a 10,000-element array and a ForkJoinPool, invoking the tasks to compute the initial risk, apply the stress test, and recalculate the risk. The terminal output verified correct execution, displaying the initial risk score, the successful stress test completion, and the increased post-stress score.

2. Code

File: *ParallelRiskCalculator.java*



File: *PortfolioThresholdTask.java*



```java
import java.util.concurrent.RecursiveAction;

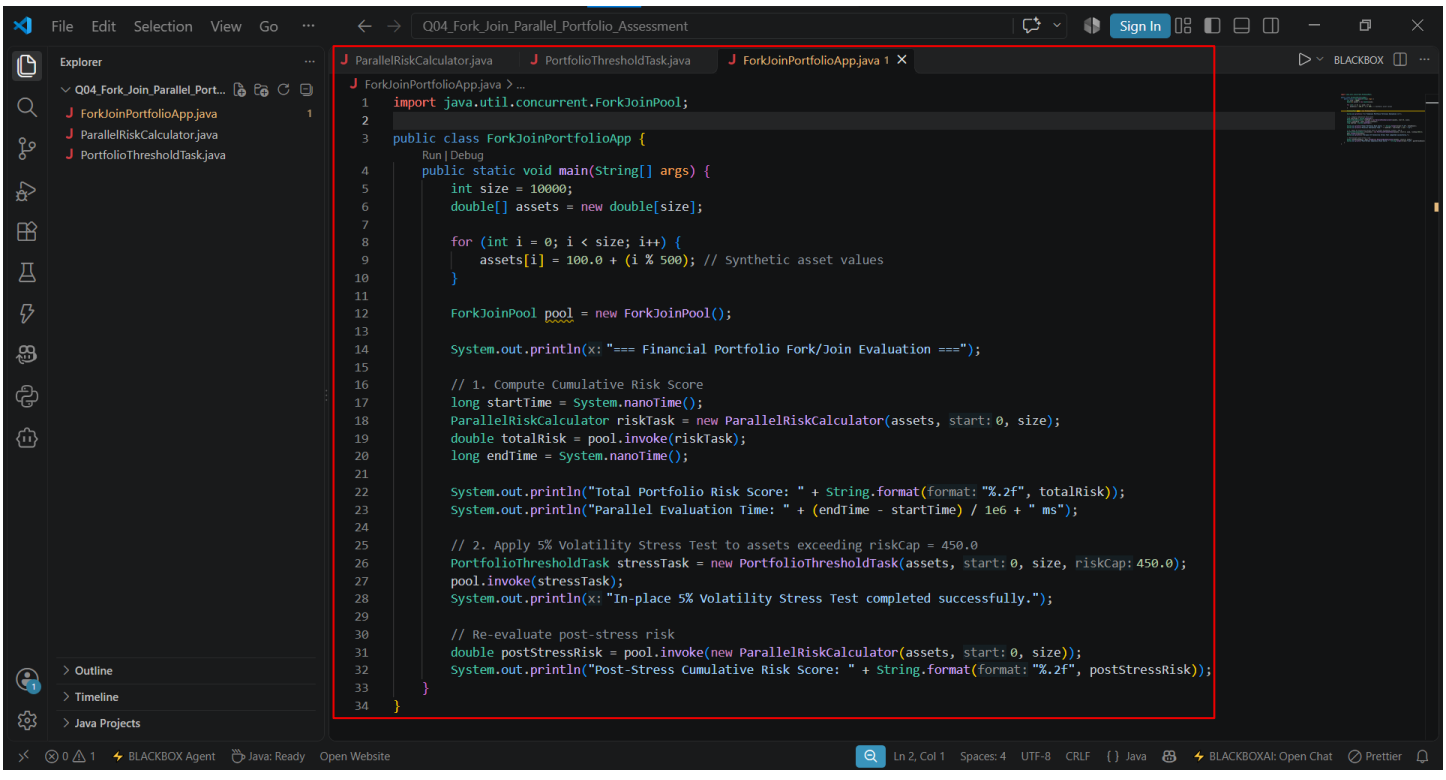
public class PortfolioThresholdTask extends RecursiveAction {
    private static final int THRESHOLD = 1000;
    private double[] assets;
    private int start;
    private int end;
    private double riskCap;

    public PortfolioThresholdTask(double[] assets, int start, int end, double riskCap) {
        this.assets = assets;
        this.start = start;
        this.end = end;
        this.riskCap = riskCap;
    }

    @Override
    protected void compute() {
        int length = end - start;
        if (length <= THRESHOLD) {
            // In-place 5% penalty adjustment on values exceeding riskCap
            for (int i = start; i < end; i++) {
                if (assets[i] > riskCap) {
                    assets[i] += assets[i] * 0.05;
                }
            }
        } else {
            int mid = start + (length / 2);
            PortfolioThresholdTask left = new PortfolioThresholdTask(assets, start, mid, riskCap);
            PortfolioThresholdTask right = new PortfolioThresholdTask(assets, mid, end, riskCap);

            left.fork();
            right.compute();
            left.join();
        }
    }
}
```

File: *ForkJoinPortfolioApp.java*



```java
import java.util.concurrent.ForkJoinPool;

public class ForkJoinPortfolioApp {
    Run | Debug
    public static void main(String[] args) {
        int size = 10000;
        double[] assets = new double[size];

        for (int i = 0; i < size; i++) {
            assets[i] = 100.0 + (i % 500); // Synthetic asset values
        }

        ForkJoinPool pool = new ForkJoinPool();

        System.out.println(x: "=== Financial Portfolio Fork/Join Evaluation ===");

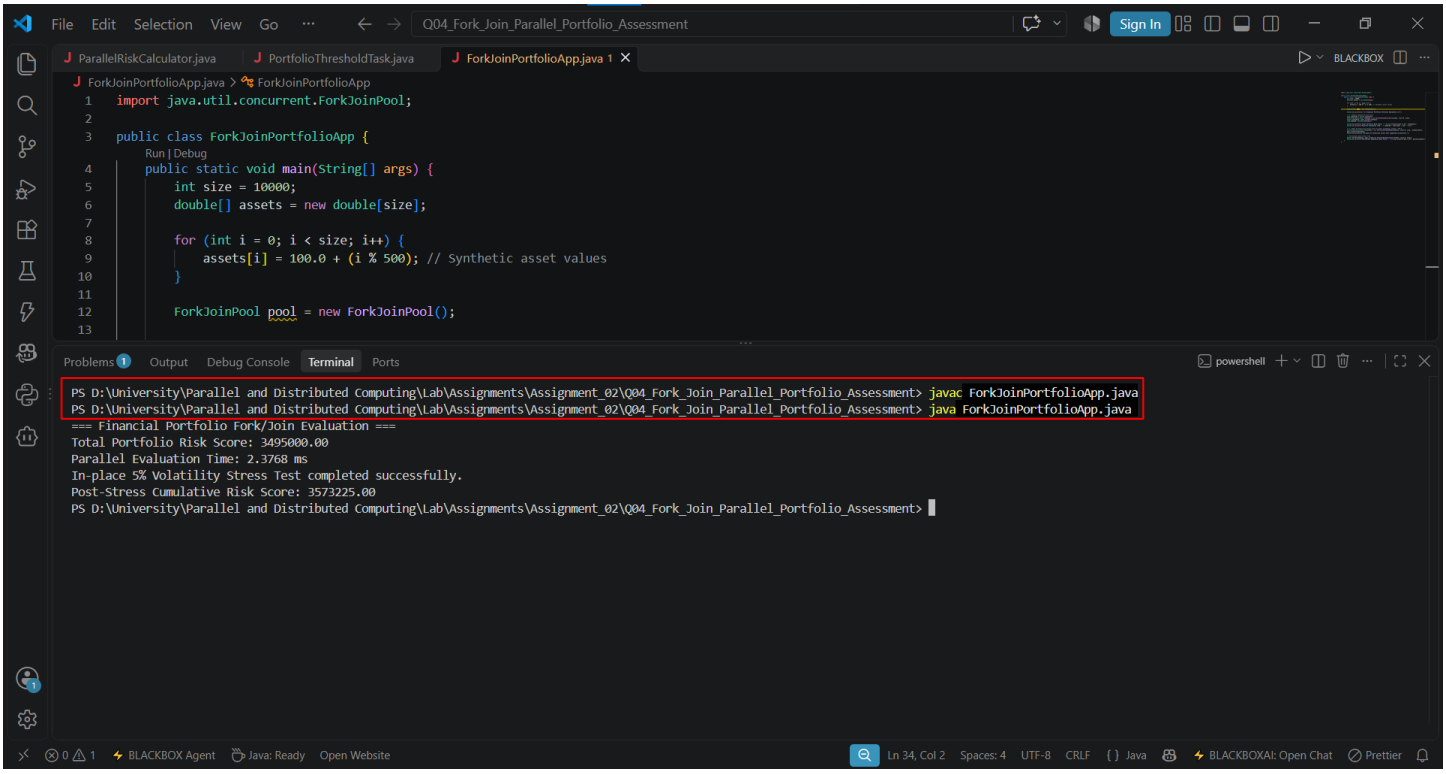
        // 1. Compute Cumulative Risk Score
        long startTime = System.nanoTime();
        ParallelRiskCalculator riskTask = new ParallelRiskCalculator(assets, start: 0, size);
        double totalRisk = pool.invoke(riskTask);
        long endTime = System.nanoTime();

        System.out.println("Total Portfolio Risk Score: " + String.format(format: "%.2f", totalRisk));
        System.out.println("Parallel Evaluation Time: " + (endTime - startTime) / 1e6 + " ms");

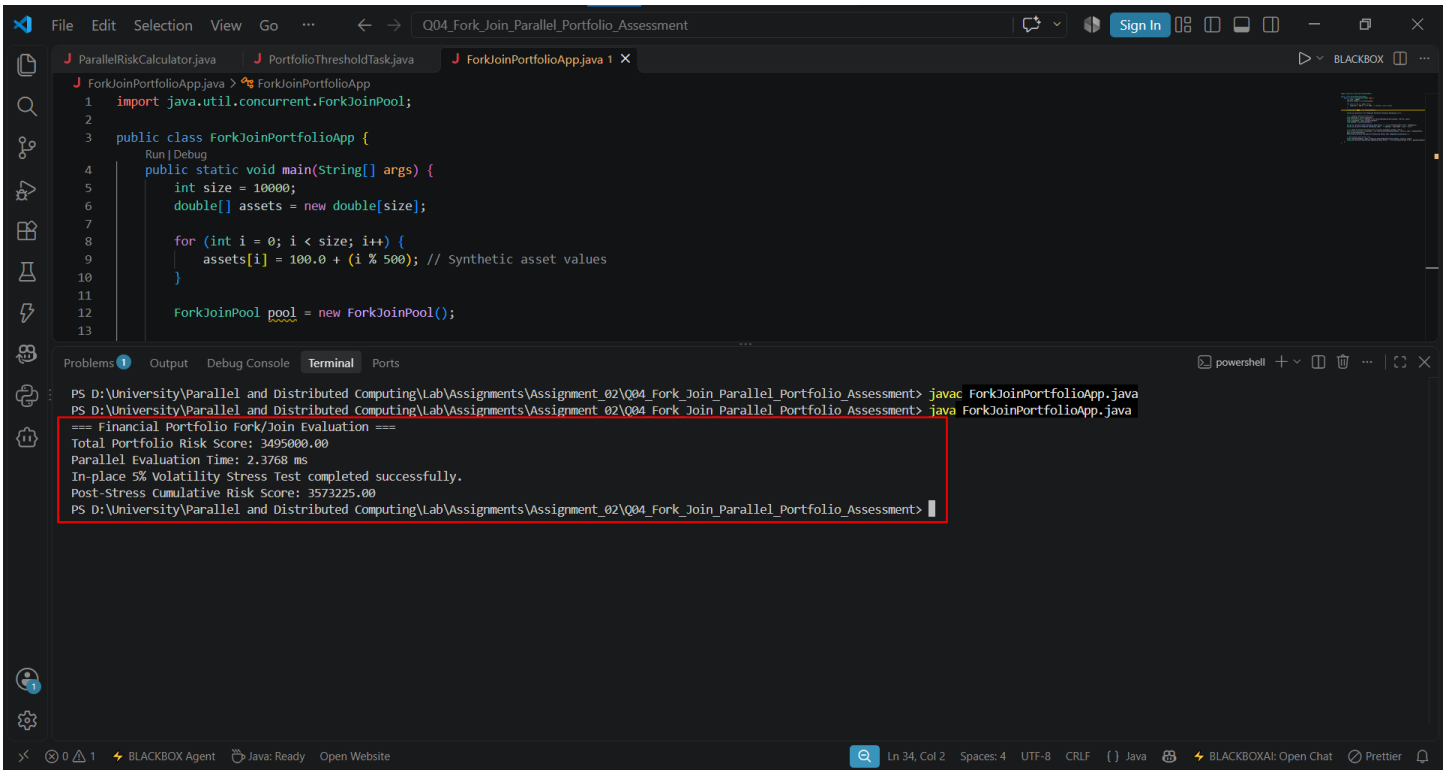
        // 2. Apply 5% Volatility Stress Test to assets exceeding riskCap = 450.0
        PortfolioThresholdTask stressTask = new PortfolioThresholdTask(assets, start: 0, size, riskCap: 450.0);
        pool.invoke(stressTask);
        System.out.println(x: "In-place 5% Volatility Stress Test completed successfully.");

        // Re-evaluate post-stress risk
        double postStressRisk = pool.invoke(new ParallelRiskCalculator(assets, start: 0, size));
        System.out.println("Post-Stress Cumulative Risk Score: " + String.format(format: "%.2f", postStressRisk));
    }
}
```

3. Compilation & Execution Screenshot



4. Output Screenshot



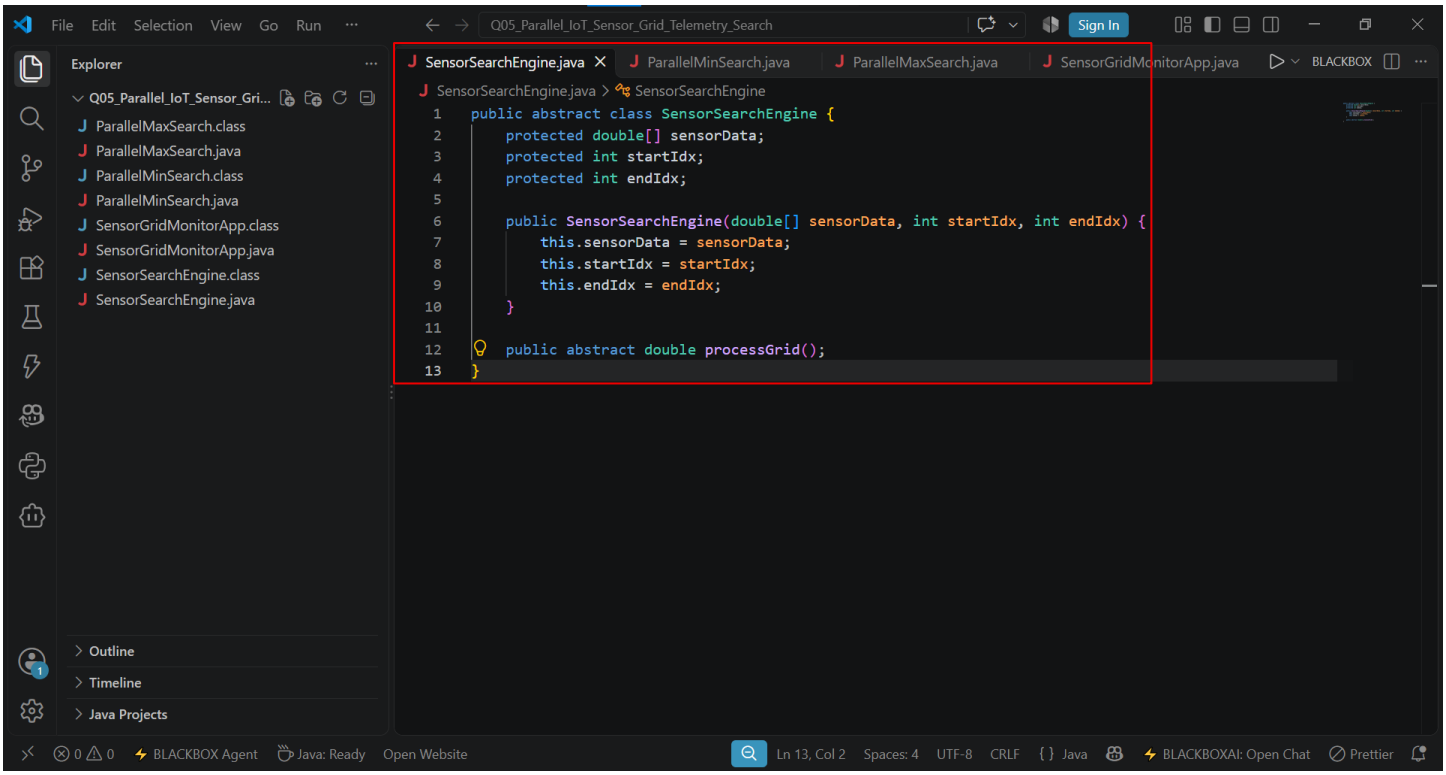
Question 5: Parallel Sensor Search Engine (Abstract Class & Threads)

1. Approach / Explanation

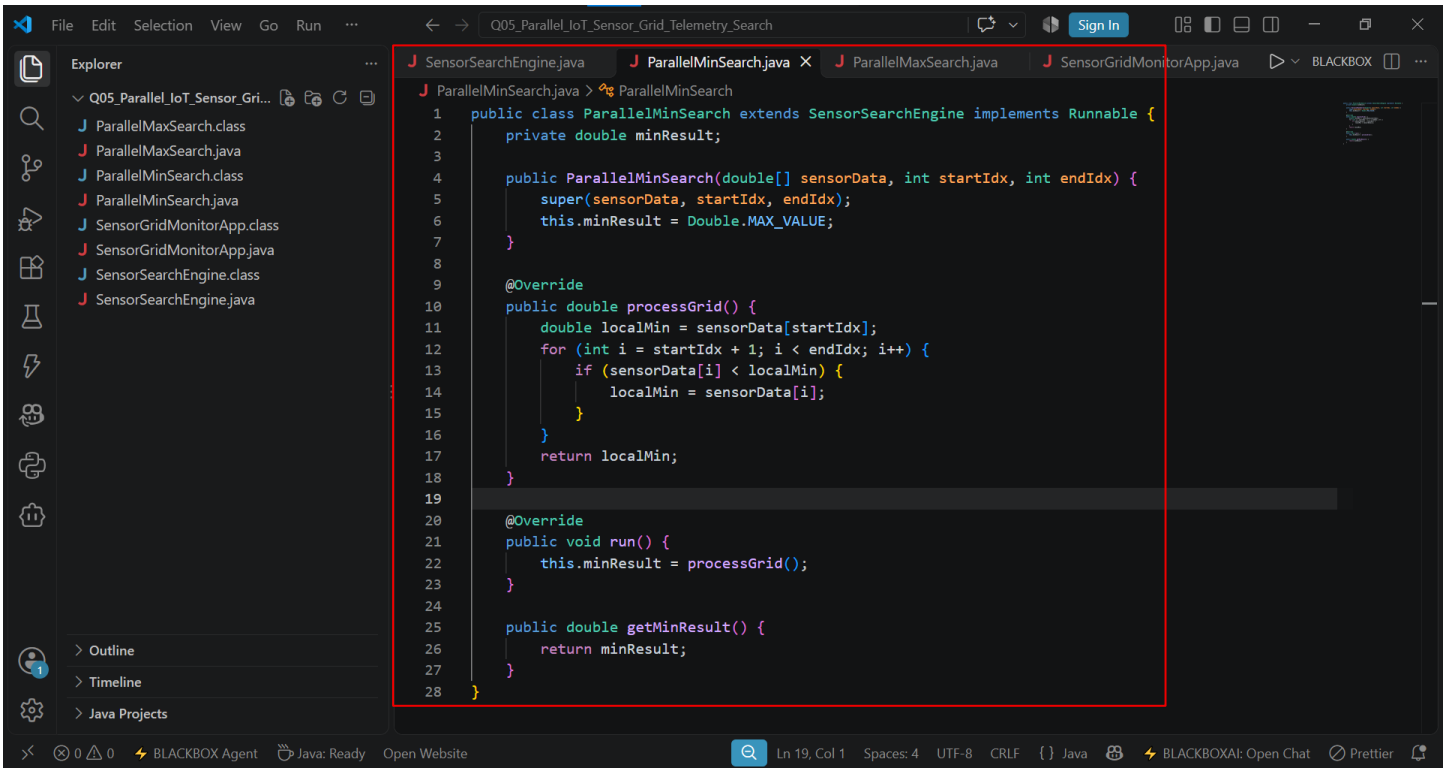
I created an abstract base class SensorSearchEngine and two concrete Runnable implementations (ParallelMinSearch and ParallelMaxSearch) to build a parallel IoT telemetry search engine. I used inheritance and abstraction to share the sensorData array and index boundaries across subclasses, which then overrode the abstract processGrid method with for loops to scan for local minimums and maximums. In the SensorGridMonitorApp main method, I partitioned a 50,000-element array into four segments and spawned eight threads concurrently (four for min, four for max) using Thread.start(). To ensure accurate aggregation, I used Thread.join() so the main thread would wait for all workers to finish before reducing their local results into global min and max values. The terminal output verified correct execution by successfully identifying the injected extreme anomalies (-12.4 °C and 198.8 Hz) and displaying the total partitioned search time.

2. Code

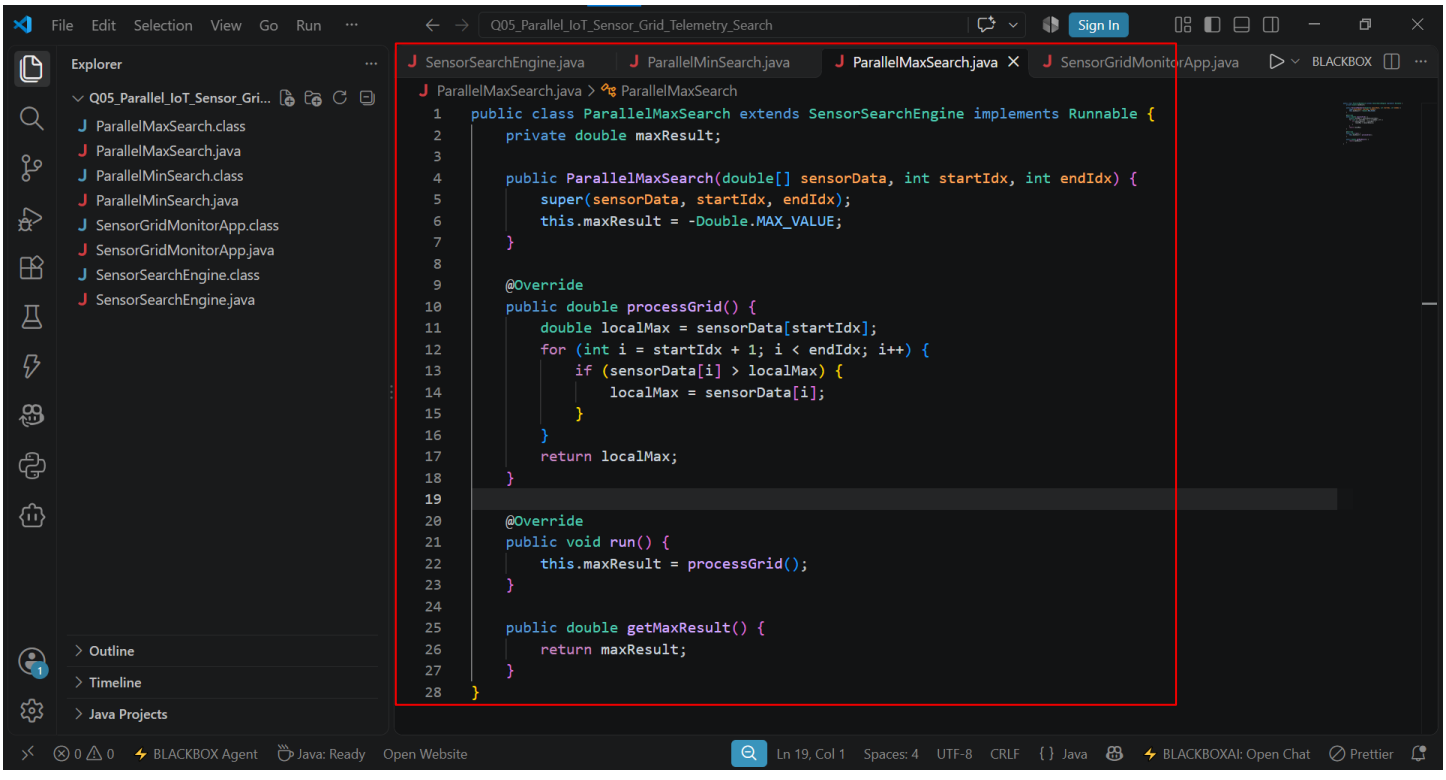
File: *SensorSearchEngine.java*



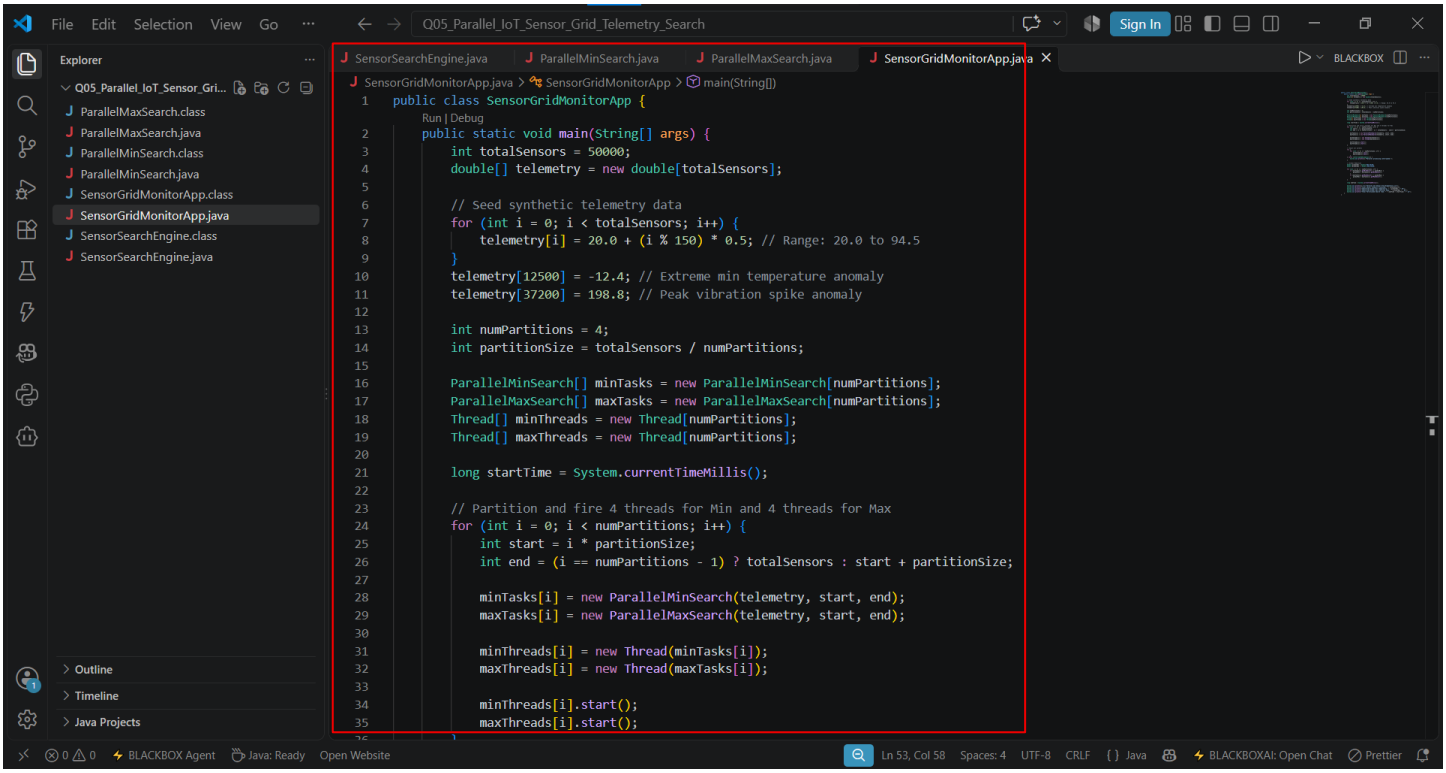
File: *ParallelMinSearch.java*



File: *ParallelMaxSearch.java*



File: *SensorGridMonitorApp.java*



```java
public class SensorGridMonitorApp {
    Run | Debug
    public static void main(String[] args) {
        int totalSensors = 50000;
        double[] telemetry = new double[totalSensors];

        // Seed synthetic telemetry data
        for (int i = 0; i < totalSensors; i++) {
            telemetry[i] = 20.0 + (i % 150) * 0.5; // Range: 20.0 to 94.5
        }
        telemetry[12500] = -12.4; // Extreme min temperature anomaly
        telemetry[37200] = 198.8; // Peak vibration spike anomaly

        int numPartitions = 4;
        int partitionSize = totalSensors / numPartitions;

        ParallelMinSearch[] minTasks = new ParallelMinSearch[numPartitions];
        ParallelMaxSearch[] maxTasks = new ParallelMaxSearch[numPartitions];
        Thread[] minThreads = new Thread[numPartitions];
        Thread[] maxThreads = new Thread[numPartitions];

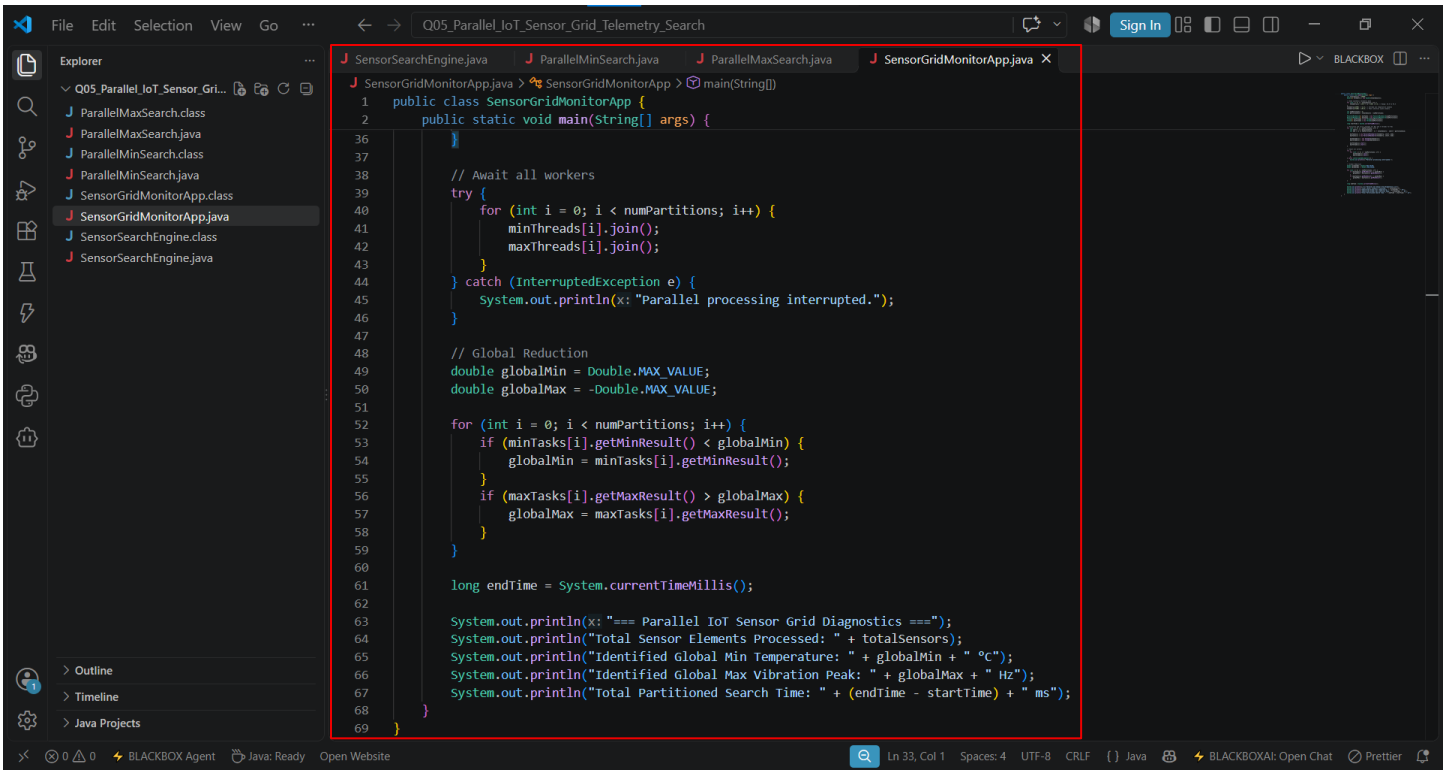
        long startTime = System.currentTimeMillis();

        // Partition and fire 4 threads for Min and 4 threads for Max
        for (int i = 0; i < numPartitions; i++) {
            int start = i * partitionSize;
            int end = (i == numPartitions - 1) ? totalSensors : start + partitionSize;

            minTasks[i] = new ParallelMinSearch(telemetry, start, end);
            maxTasks[i] = new ParallelMaxSearch(telemetry, start, end);

            minThreads[i] = new Thread(minTasks[i]);
            maxThreads[i] = new Thread(maxTasks[i]);

            minThreads[i].start();
            maxThreads[i].start();
        }
```



```java
public class SensorGridMonitorApp {
    public static void main(String[] args) {
        }

        // Await all workers
        try {
            for (int i = 0; i < numPartitions; i++) {
                minThreads[i].join();
                maxThreads[i].join();
            }
        } catch (InterruptedException e) {
            System.out.println(x: "Parallel processing interrupted.");
        }

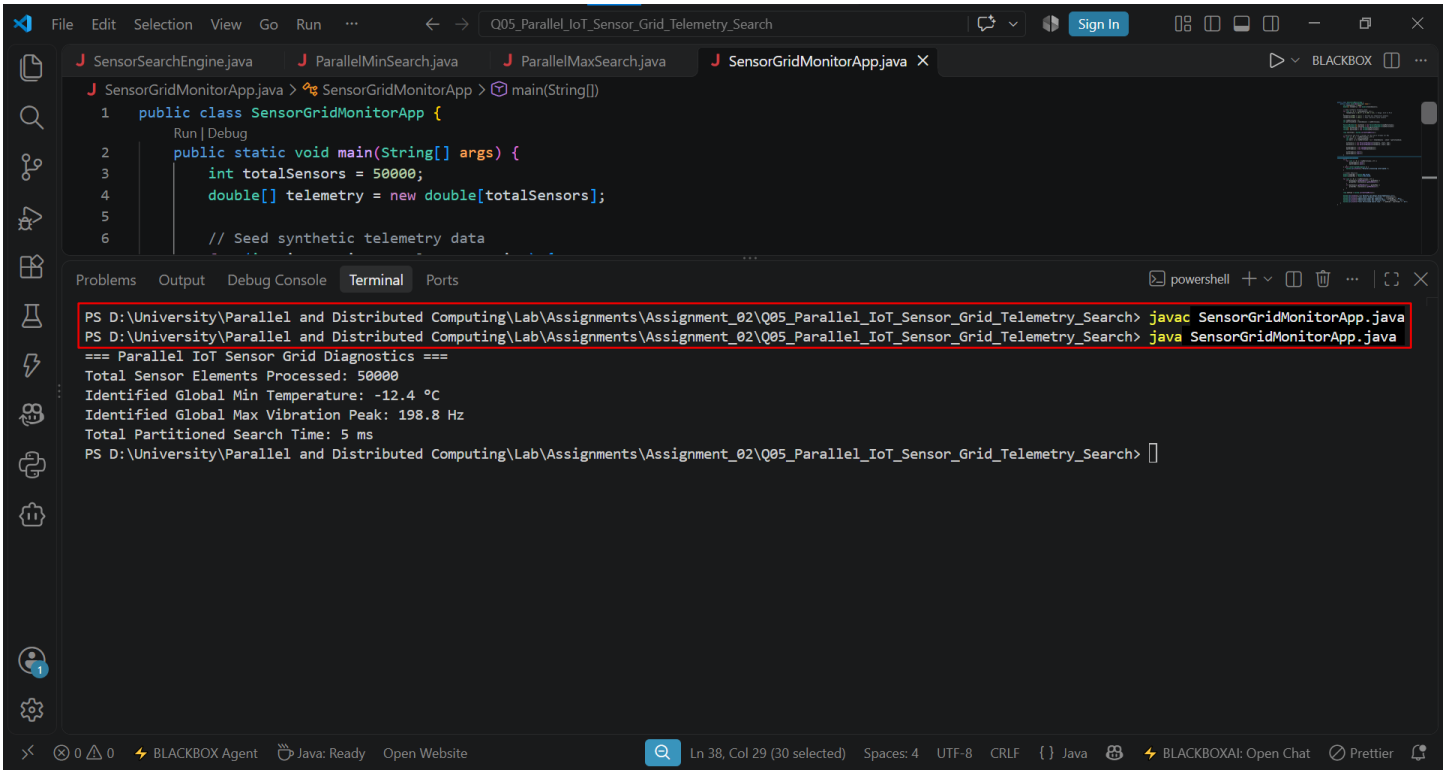
        // Global Reduction
        double globalMin = Double.MAX_VALUE;
        double globalMax = -Double.MAX_VALUE;

        for (int i = 0; i < numPartitions; i++) {
            if (minTasks[i].getMinResult() < globalMin) {
                globalMin = minTasks[i].getMinResult();
            }
            if (maxTasks[i].getMaxResult() > globalMax) {
                globalMax = maxTasks[i].getMaxResult();
            }
        }

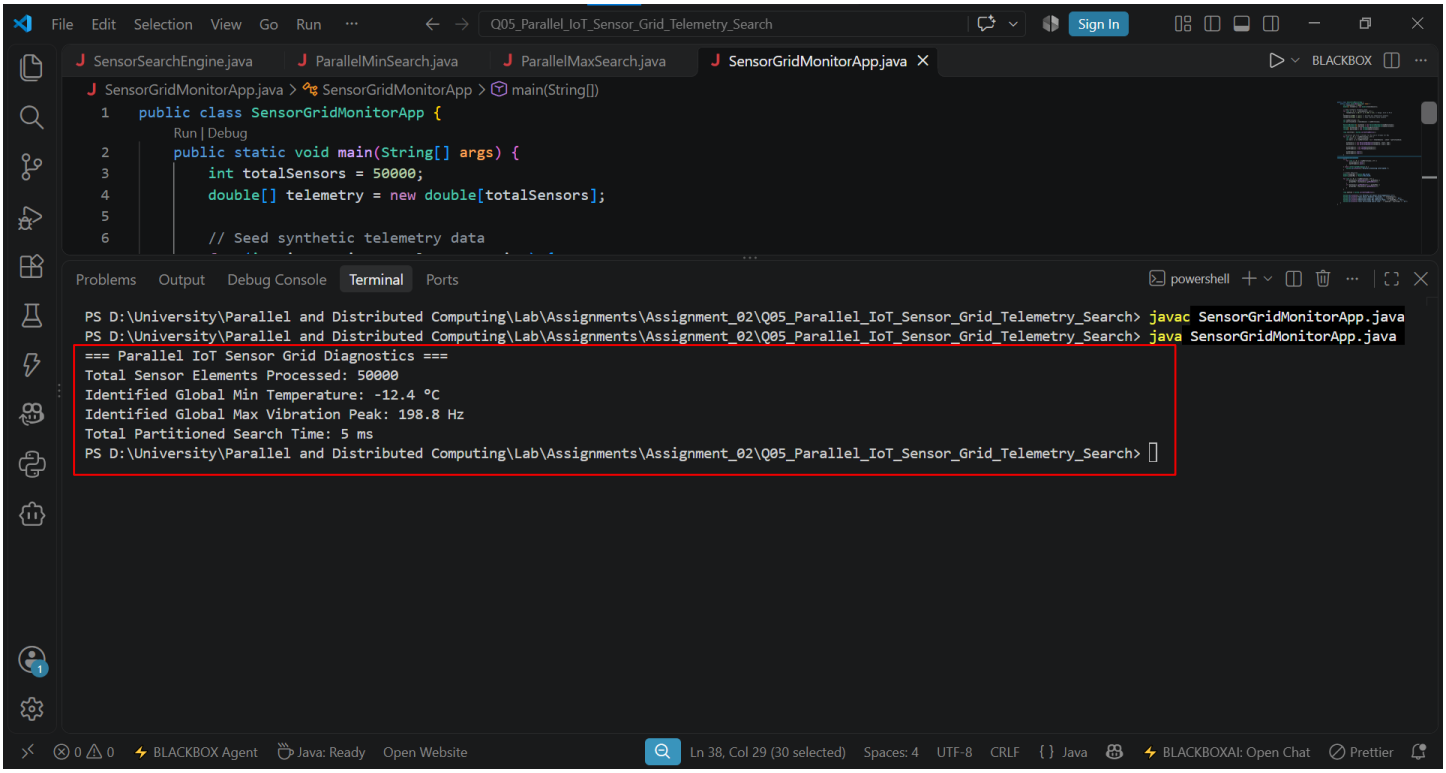
        long endTime = System.currentTimeMillis();

        System.out.println(x: "=== Parallel IoT Sensor Grid Diagnostics ===");
        System.out.println("Total Sensor Elements Processed: " + totalSensors);
        System.out.println("Identified Global Min Temperature: " + globalMin + " °C");
        System.out.println("Identified Global Max Vibration Peak: " + globalMax + " Hz");
        System.out.println("Total Partitioned Search Time: " + (endTime - startTime) + " ms");
    }
}
```

3. Compilation & Execution Screenshot



4. Output Screenshot



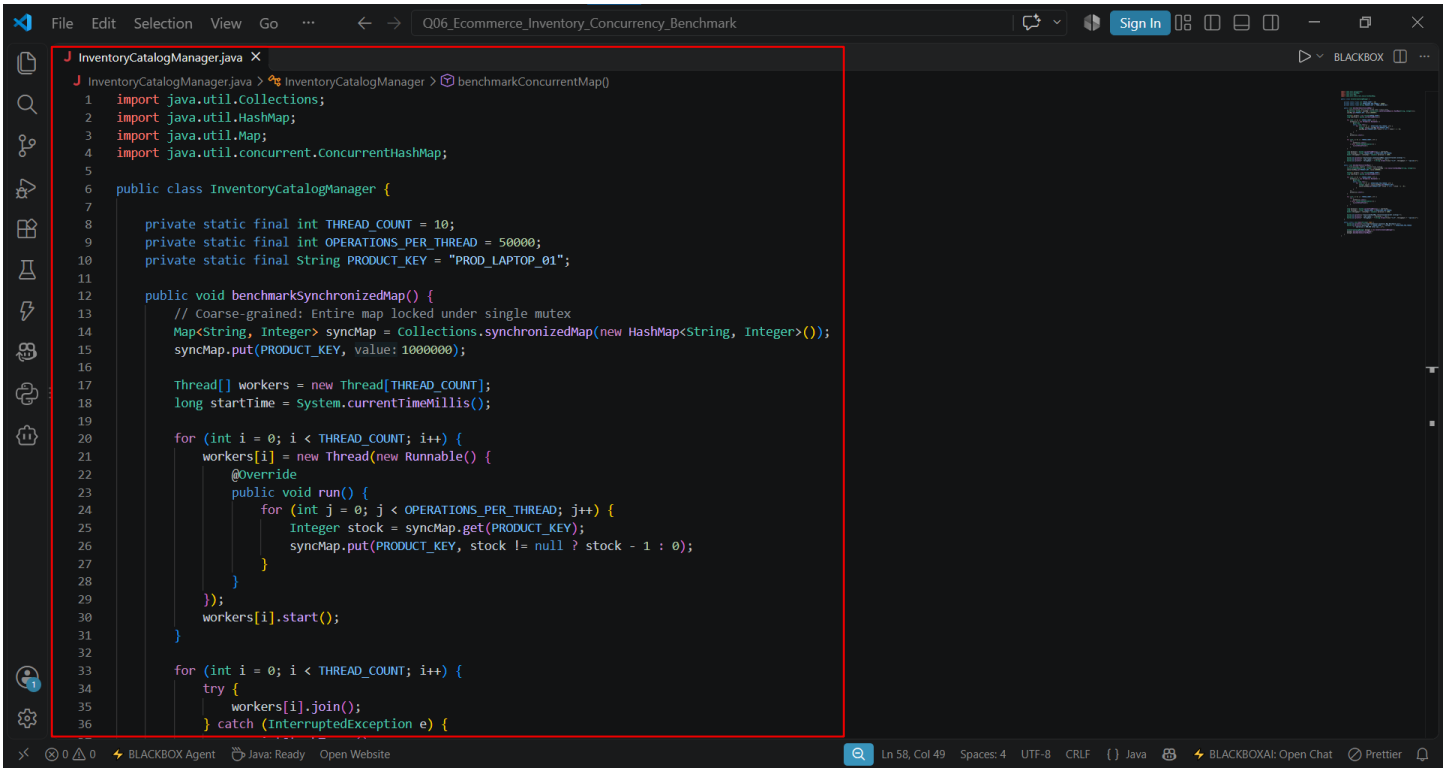
Question 6: Inventory Catalog Concurrency Benchmark (Synchronized vs Concurrent Map)

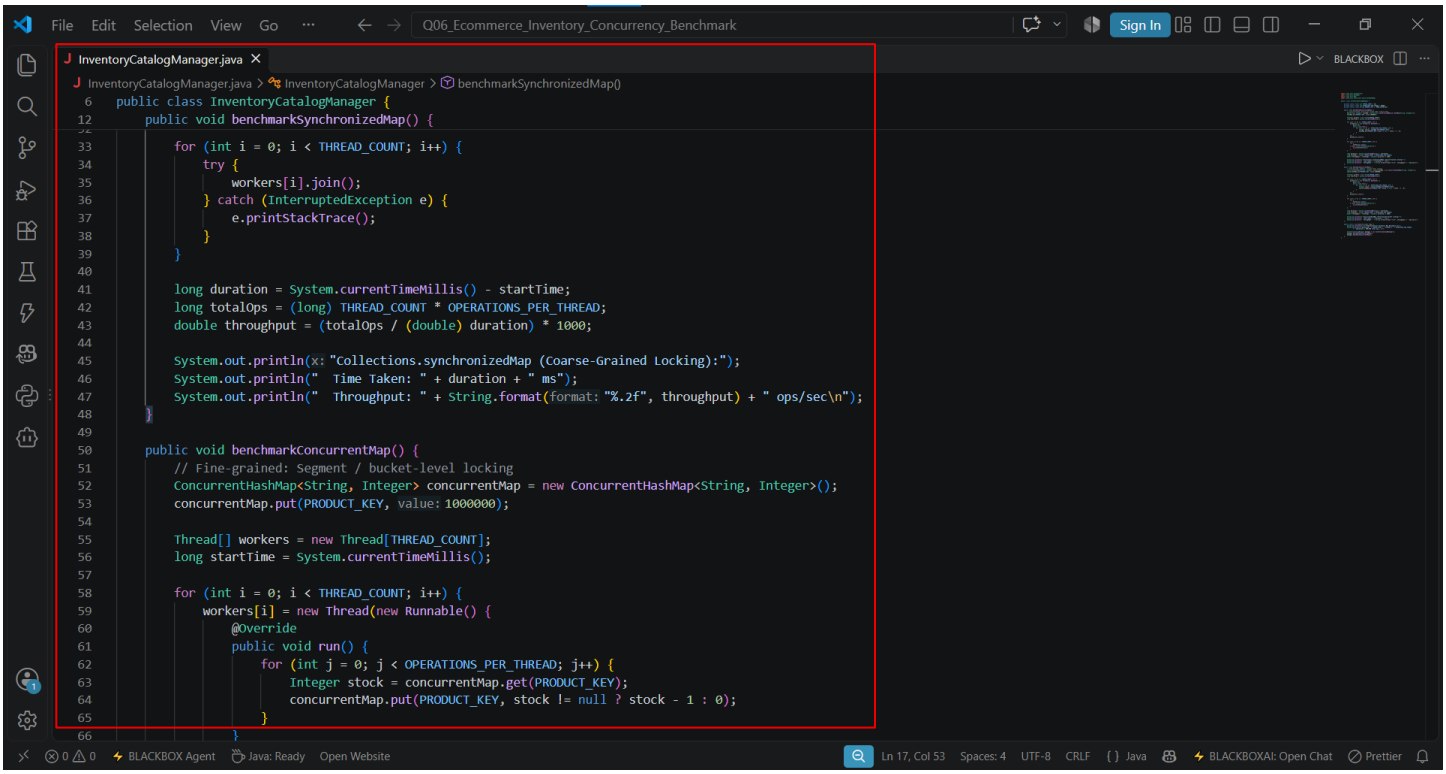
1. Approach / Explanation

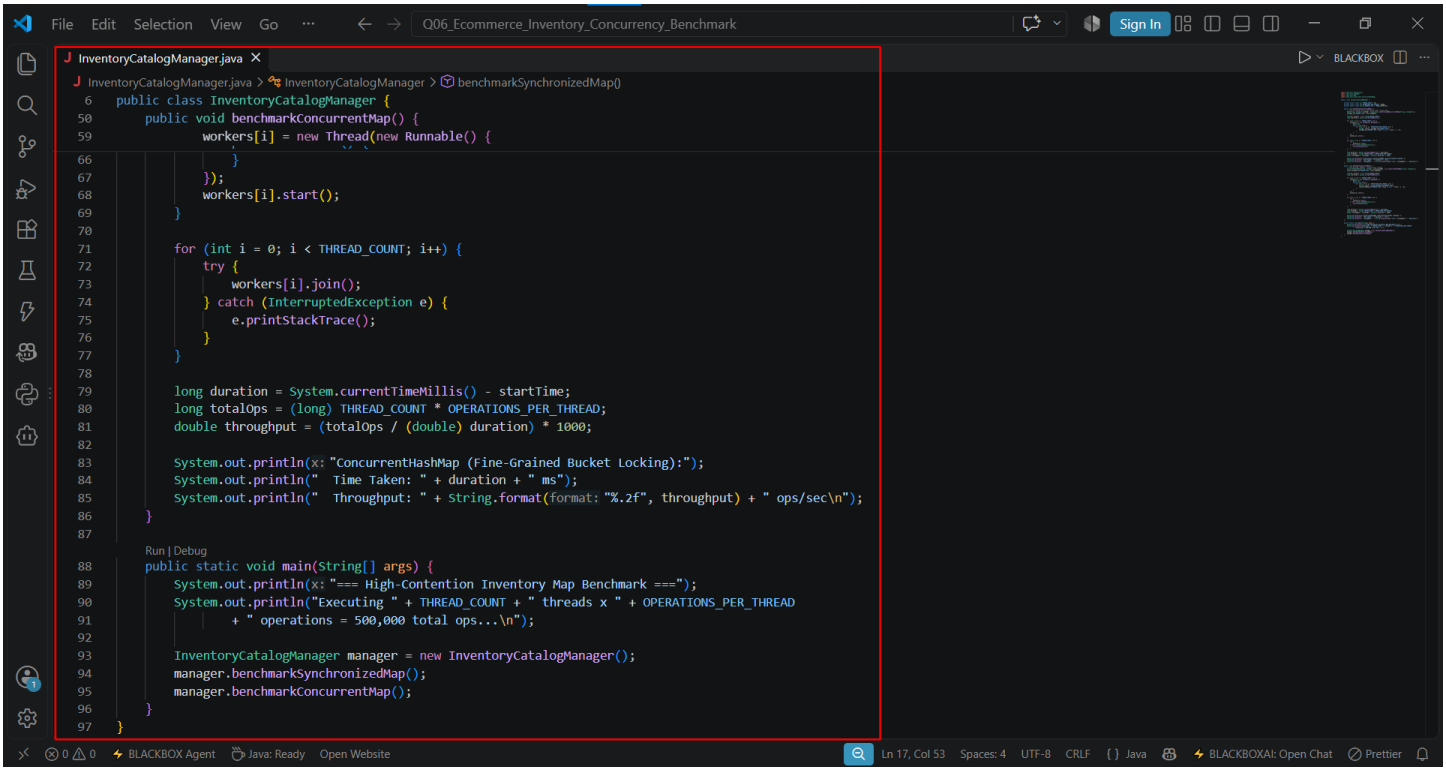
I created InventoryCatalogManager.java to benchmark the performance of two thread-safe map implementations under high contention. I implemented two methods that spawn ten Runnable threads using anonymous inner classes, each executing a for loop to perform 50,000 rapid get and put operations on a shared inventory key. To measure concurrency overhead, the first method used Collections.synchronizedMap (coarse-grained locking), while the second used ConcurrentHashMap (fine-grained bucket locking), utilizing Thread.join() to synchronize the completion of all threads before calculating execution time and throughput. The terminal output verified the architectural advantage of ConcurrentHashMap, which processed the workload in 83 ms (over 6 million ops/sec) compared to the synchronized map's 135 ms (3.7 million ops/sec).

2. Code

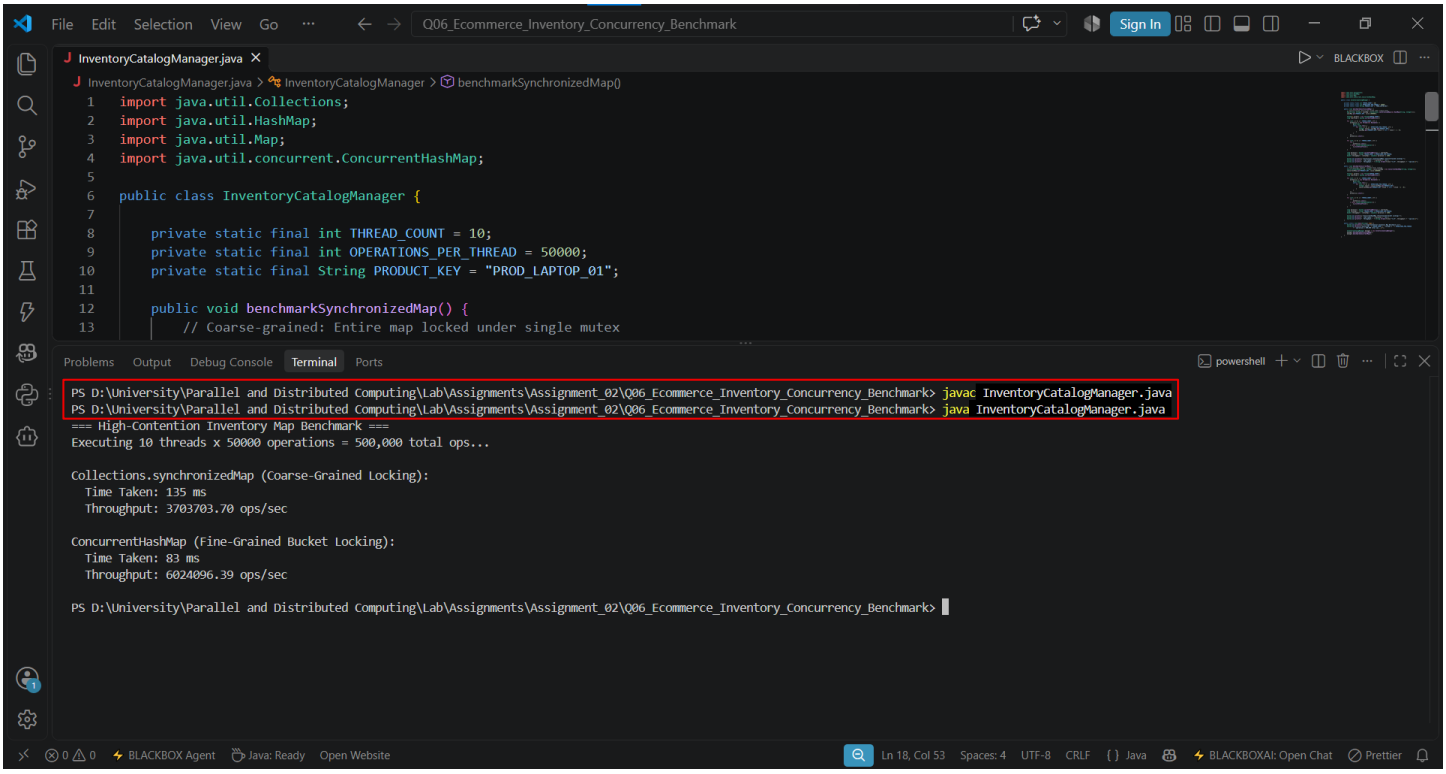
File: *InventoryCatalogManager.java*



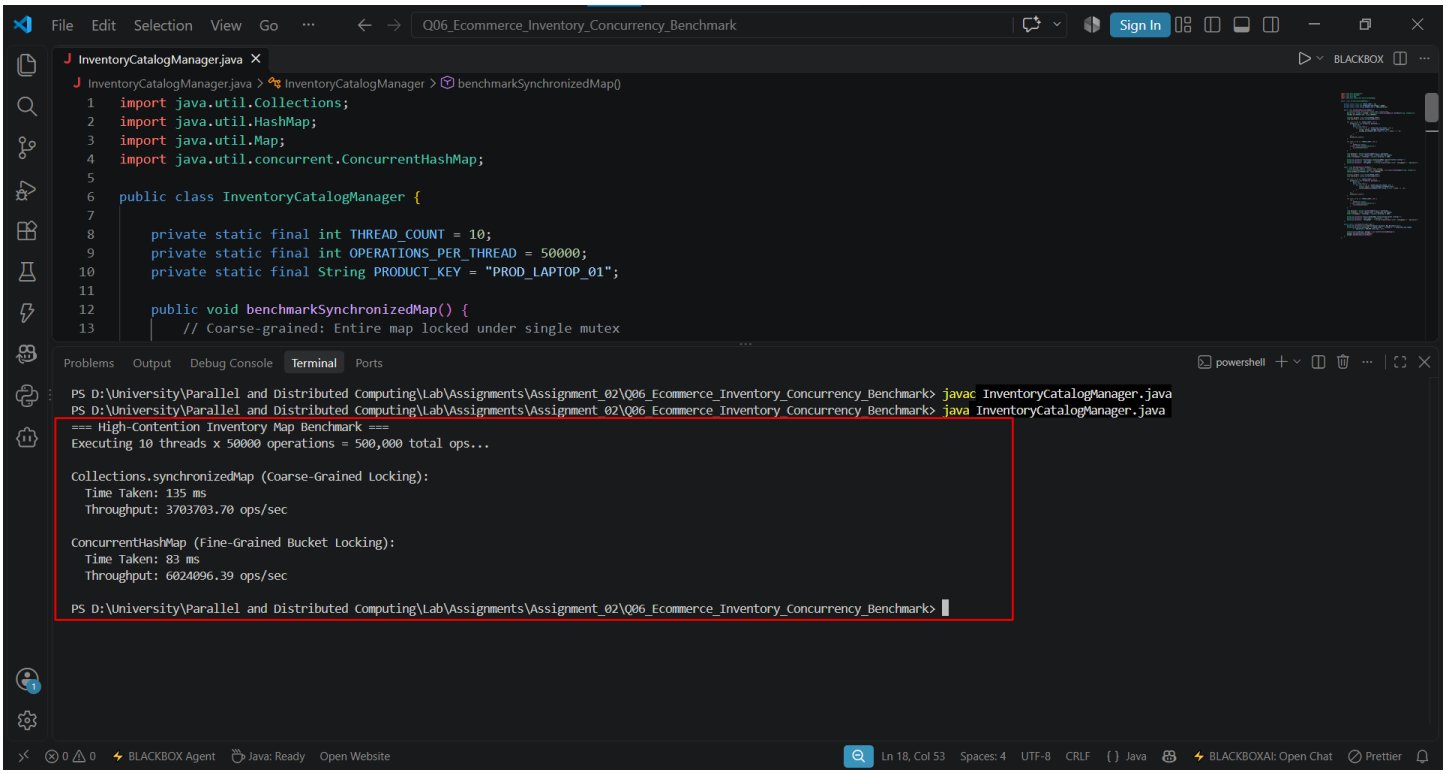

File Edit Selection View Go ···
Q06_Ecommerce_Inventory_Concurrency_Benchmark
Sign In
BLACKBOX
InventoryCatalogManager.java
InventoryCatalogManager.java > InventoryCatalogManager > benchmarkSynchronizedMap()
6 public class InventoryCatalogManager {
12 public void benchmarkSynchronizedMap() {
33 for (int i = 0; i < THREAD_COUNT; i++) {
34 try {
35 workers[i].join();
36 } catch (InterruptedException e) {
37 e.printStackTrace();
38 }
39 }
40
41 long duration = System.currentTimeMillis() - startTime;
42 long totalOps = (long) THREAD_COUNT * OPERATIONS_PER_THREAD;
43 double throughput = (totalOps / (double) duration) * 1000;
44
45 System.out.println(x: "Collections.synchronizedMap (Coarse-Grained Locking):");
46 System.out.println(" Time Taken: " + duration + " ms");
47 System.out.println(" Throughput: " + String.format(format: "%.2f", throughput) + " ops/sec\n");
48 }
49
50 public void benchmarkConcurrentMap() {
51 // Fine-grained: Segment / bucket-level locking
52 ConcurrentHashMap<String, Integer> concurrentMap = new ConcurrentHashMap<String, Integer>();
53 concurrentMap.put(PRODUCT_KEY, value: 1000000);
54
55 Thread[] workers = new Thread[THREAD_COUNT];
56 long startTime = System.currentTimeMillis();
57
58 for (int i = 0; i < THREAD_COUNT; i++) {
59 workers[i] = new Thread(new Runnable() {
60 @Override
61 public void run() {
62 for (int j = 0; j < OPERATIONS_PER_THREAD; j++) {
63 Integer stock = concurrentMap.get(PRODUCT_KEY);
64 concurrentMap.put(PRODUCT_KEY, stock != null ? stock - 1 : 0);
65 }
66 }
BLACKBOX Agent
Java: Ready
Open Website
Ln 17, Col 53
Spaces: 4
UTF-8
CRLF
{ } Java
BLACKBOXAI: Open Chat
Prettier


File Edit Selection View Go ···
Q06_Ecommerce_Inventory_Concurrency_Benchmark
Sign In
BLACKBOX
InventoryCatalogManager.java
InventoryCatalogManager.java > InventoryCatalogManager > benchmarkSynchronizedMap()
6 public class InventoryCatalogManager {
50 public void benchmarkConcurrentMap() {
59 workers[i] = new Thread(new Runnable() {
66 }
67 });
68 workers[i].start();
69 }
70
71 for (int i = 0; i < THREAD_COUNT; i++) {
72 try {
73 workers[i].join();
74 } catch (InterruptedException e) {
75 e.printStackTrace();
76 }
77 }
78
79 long duration = System.currentTimeMillis() - startTime;
80 long totalOps = (long) THREAD_COUNT * OPERATIONS_PER_THREAD;
81 double throughput = (totalOps / (double) duration) * 1000;
82
83 System.out.println(x: "ConcurrentHashMap (Fine-Grained Bucket Locking):");
84 System.out.println(" Time Taken: " + duration + " ms");
85 System.out.println(" Throughput: " + String.format(format: "%.2f", throughput) + " ops/sec\n");
86 }
87
Run | Debug
88 public static void main(String[] args) {
89 System.out.println(x: "=== High-Contention Inventory Map Benchmark ===");
90 System.out.println("Executing " + THREAD_COUNT + " threads x " + OPERATIONS_PER_THREAD
91 + " operations = 500,000 total ops...\n");
92
93 InventoryCatalogManager manager = new InventoryCatalogManager();
94 manager.benchmarkSynchronizedMap();
95 manager.benchmarkConcurrentMap();
96 }
97 }
BLACKBOX Agent
Java: Ready
Open Website
Ln 17, Col 53
Spaces: 4
UTF-8
CRLF
{ } Java
BLACKBOXAI: Open Chat
Prettier

3. Compilation & Execution Screenshot



4. Output Screenshot



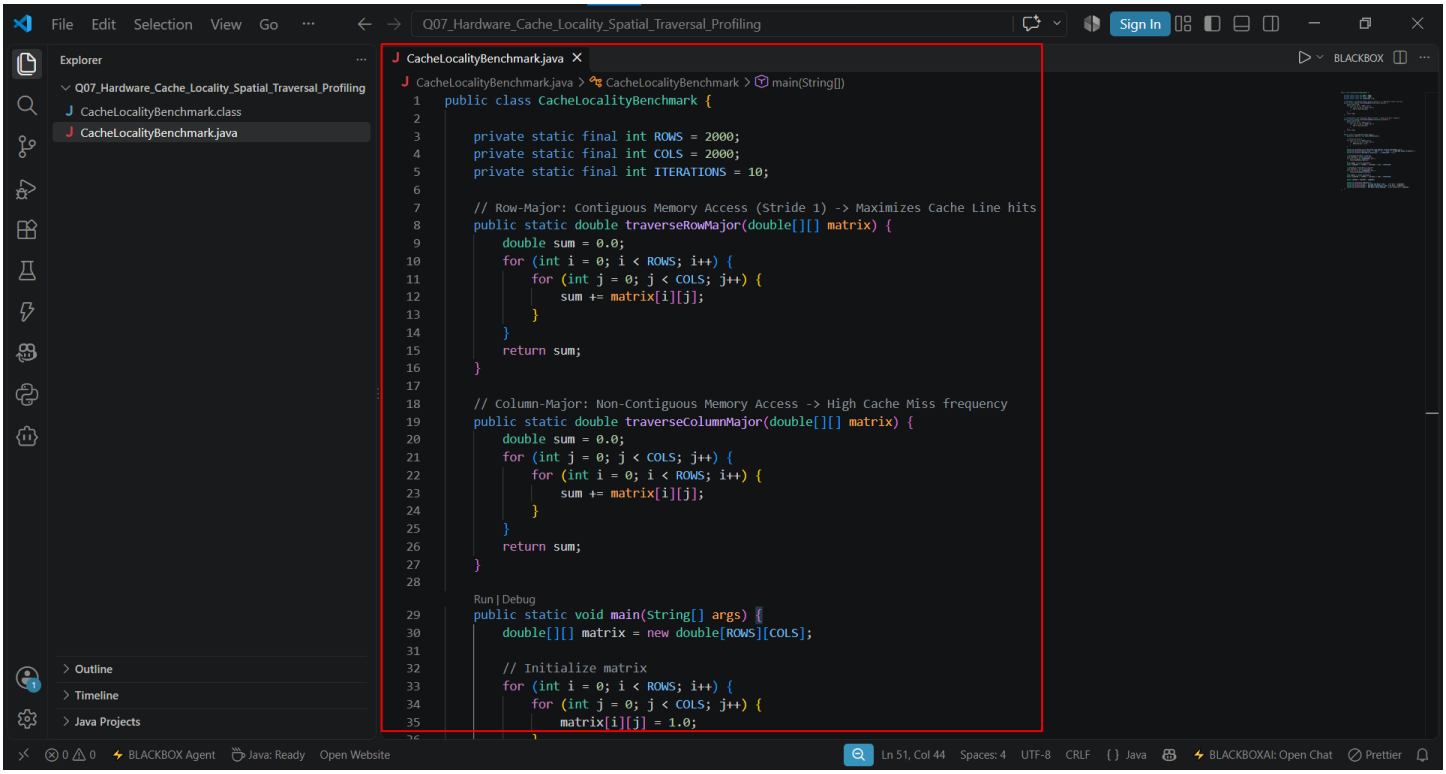
Question 7: CPU Cache Locality Benchmark (Row-Major vs Column-Major)

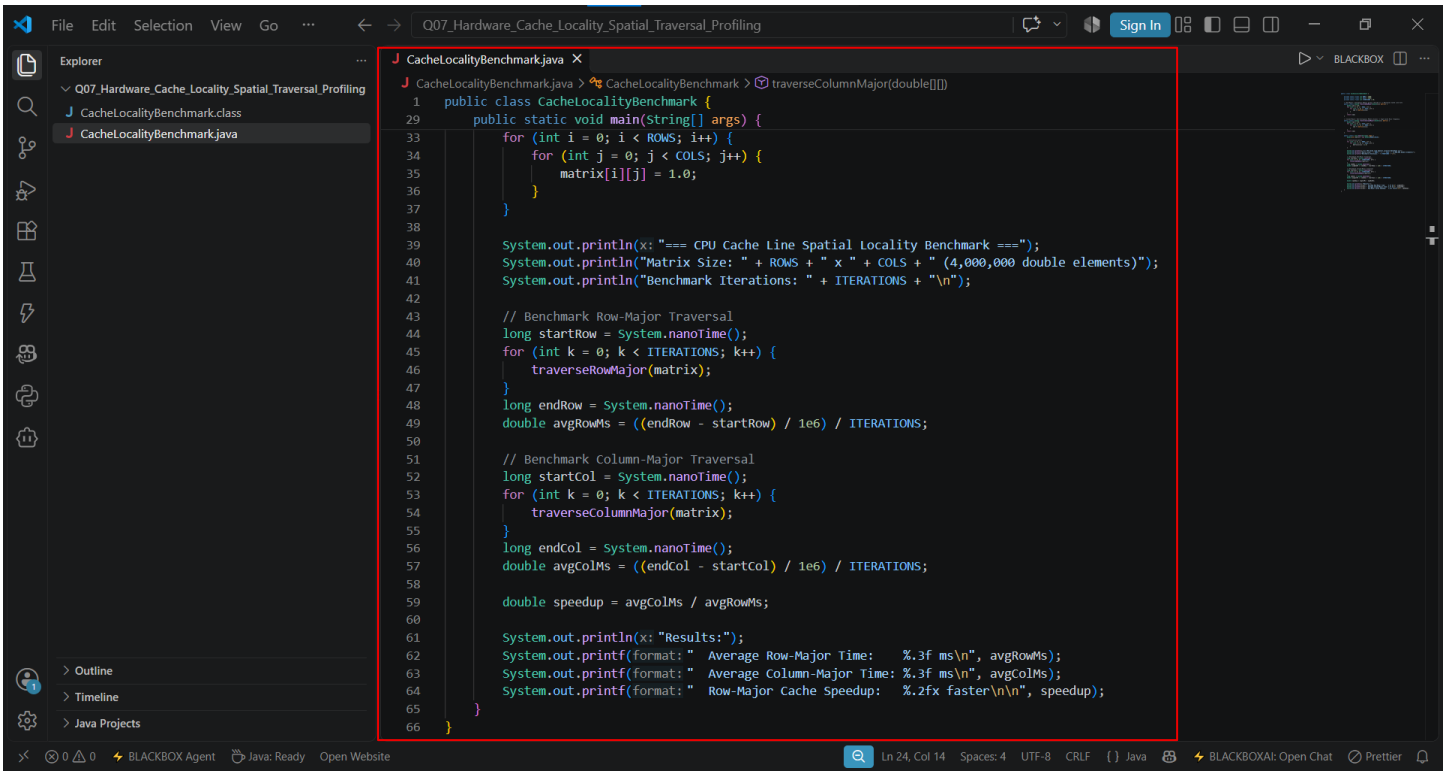
1. Approach / Explanation

I created CacheLocalityBenchmark.java to demonstrate how spatial locality and CPU cache lines affect memory access speeds. I initialized a 2000x2000 double array and implemented two methods: traverseRowMajor using a standard [i][j] nested for loop for contiguous memory access, and traverseColumnMajor using [j][i] loops to force memory jumps across rows. In the main method, I used System.nanoTime() to measure the total time for 10 iterations of each traversal and calculated the average execution time and speedup factor. The terminal output verified the architectural theory, revealing that the row-major traversal (5.252 ms) was 5.69 times faster than the column-major traversal (29.905 ms) due to optimal L1 cache hits versus constant cache misses.

2. Code

File: *CacheLocalityBenchmark.java*





```java
public class CacheLocalityBenchmark {
    public static void main(String[] args) {
        for (int i = 0; i < ROWS; i++) {
            for (int j = 0; j < COLS; j++) {
                matrix[i][j] = 1.0;
            }
        }

        System.out.println("=== CPU Cache Line Spatial Locality Benchmark ===");
        System.out.println("Matrix Size: " + ROWS + " x " + COLS + " (4,000,000 double elements)");
        System.out.println("Benchmark Iterations: " + ITERATIONS + "\n");

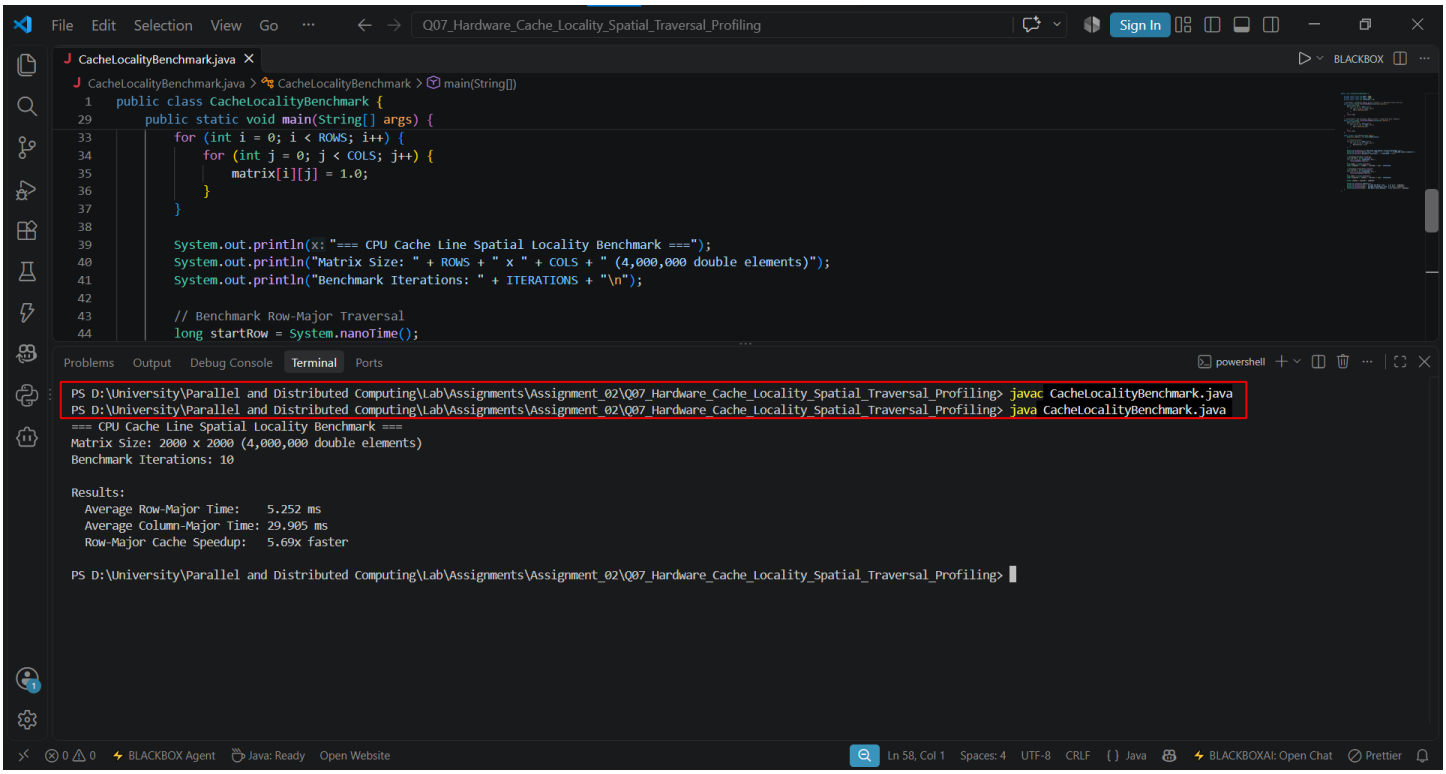
        // Benchmark Row-Major Traversal
        long startRow = System.nanoTime();
        for (int k = 0; k < ITERATIONS; k++) {
            traverseRowMajor(matrix);
        }
        long endRow = System.nanoTime();
        double avgRowMs = ((endRow - startRow) / 1e6) / ITERATIONS;

        // Benchmark Column-Major Traversal
        long startCol = System.nanoTime();
        for (int k = 0; k < ITERATIONS; k++) {
            traverseColumnMajor(matrix);
        }
        long endCol = System.nanoTime();
        double avgColMs = ((endCol - startCol) / 1e6) / ITERATIONS;

        double speedup = avgColMs / avgRowMs;

        System.out.println("Results:");
        System.out.printf("  Average Row-Major Time:    %.3f ms\n", avgRowMs);
        System.out.printf("  Average Column-Major Time: %.3f ms\n", avgColMs);
        System.out.printf("  Row-Major Cache Speedup:   %.2fx faster\n\n", speedup);
    }
}
```

3. Compilation & Execution Screenshot

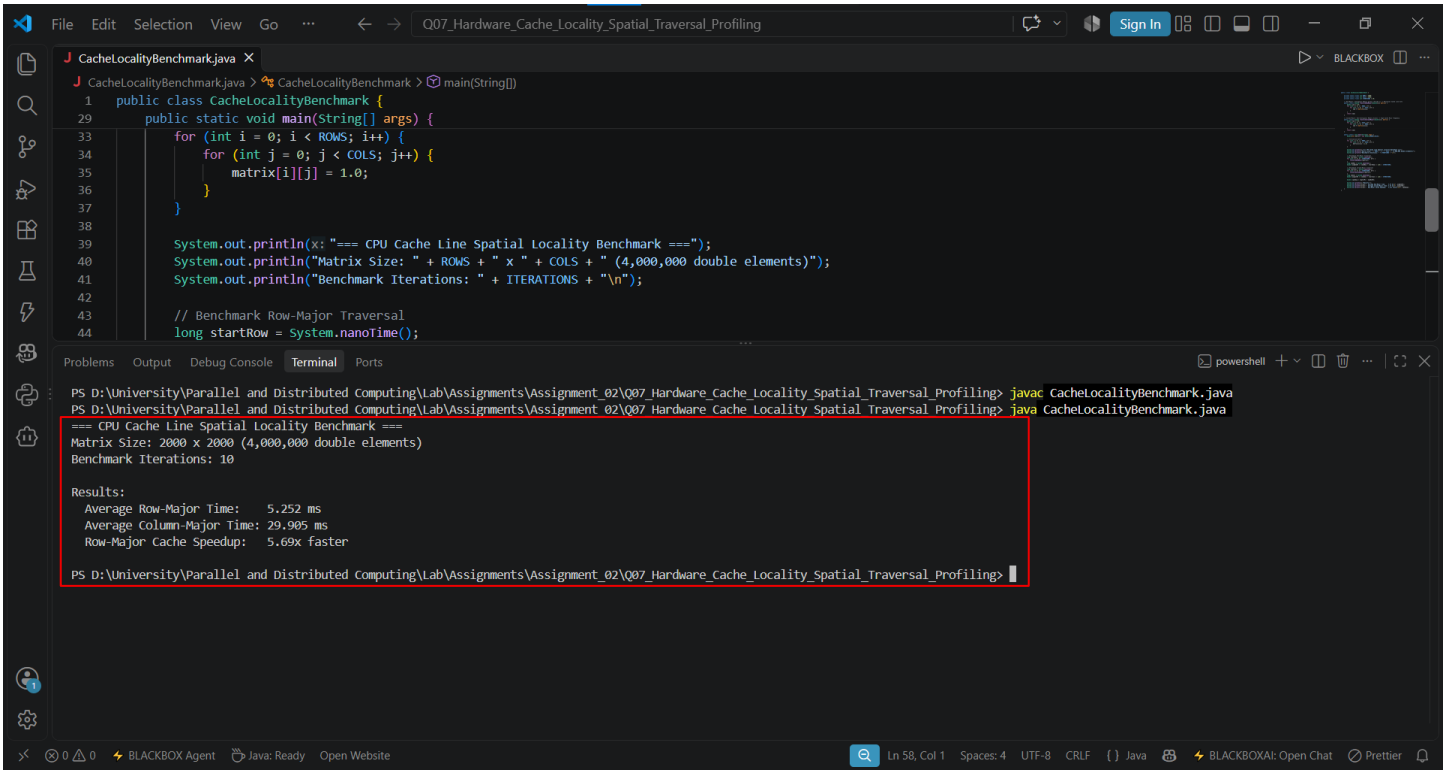


```
PS D:\University\Parallel and Distributed Computing\Lab\Assignments\Assignment_02\Q07_Hardware_Cache_Locality_Spatial_Traversal_Profiling> javac CacheLocalityBenchmark.java
PS D:\University\Parallel and Distributed Computing\Lab\Assignments\Assignment_02\Q07_Hardware_Cache_Locality_Spatial_Traversal_Profiling> java CacheLocalityBenchmark.java
=== CPU Cache Line Spatial Locality Benchmark ===
Matrix Size: 2000 x 2000 (4,000,000 double elements)
Benchmark Iterations: 10

Results:
  Average Row-Major Time:    5.252 ms
  Average Column-Major Time: 29.905 ms
  Row-Major Cache Speedup:   5.69x faster

PS D:\University\Parallel and Distributed Computing\Lab\Assignments\Assignment_02\Q07_Hardware_Cache_Locality_Spatial_Traversal_Profiling>
```

4. Output Screenshot



5. Execution Report: Cache Line Architecture & Spatial Locality

1. Cache Line Architecture: Modern CPUs read data from physical RAM in fixed 64-byte chunks known as "cache lines". A Java double requires 8 bytes; thus, loading a single element loads 8 contiguous doubles into the L1/L2 cache simultaneously.

2. Spatial Locality in Row-Major Traversal (Stride 1): In Row-Major order [i][j], elements are laid out consecutively in memory. When matrix[i][0] is accessed, the CPU hardware prefetcher loads the next 7 consecutive elements into the L1 cache. The subsequent 7 loop iterations experience instantaneous L1 cache hits without hitting system RAM.

3. Cache Miss Penalty in Column-Major Traversal: In Column-Major order [j][i], consecutive loop steps jump across entire rows (a stride of 2000 doubles = 16,000 bytes). This completely exceeds the 64-byte cache line capacity. Nearly every single read results in a cache miss, stalling the CPU execution pipeline while waiting for high-latency main memory fetches.

- End of Assignment -